SmartSchedule (AI Scheduling Assistant) — Project Proposal Report

Personal Productivity Assistant for Automated Task & Timetable Scheduling

Course: CSC10014 — Computational Thinking (2026)
Institution: Faculty of Information Technology, VNU-HCM University of Science (FIT-HCMUS)
Track: Track B — Personal Productivity Assistant
Team Name: The Seven Dwarfs (Group 03)
Project Title: SmartSchedule — Personal Productivity Assistant for Automated Task & Timetable Scheduling
Report Type: Project Proposal Report (Design Phase). Focus: **The Computational Thinking Process**

Team Information

#	Full Name	Student ID	University Email
1	**Phạm Đức Minh**	25125028	pdminh2523@apcs.fitus.edu.vn
2	**Trần Nguyên Khánh**	25125019	tnkhanh2527@apcs.fitus.edu.vn
3	**Hoàng Minh Anh**	25125001	hmanh2503@apcs.fitus.edu.vn
4	**Nguyễn Trọng Tiến**	25125037	nttien2530@apcs.fitus.edu.vn
5	**Lê Tiến Bình**	25125007	ltbinh2520@apcs.fitus.edu.vn
6	**Lê Minh Nhật**	25125032	lmnhat2532@apcs.fitus.edu.vn
7	**Nguyễn Nguyễn Gia Bảo**	25125006	nngbao2508@apcs.fitus.edu.vn

0. Summary

0.1 What we want to build

Students plan their week with static to-do lists and basic calendars. These tools do not know how long a task really takes, when the student has energy, or what to do when the plan breaks. We propose an **AI Scheduling Assistant** that takes a student's fixed commitments, tasks and preferences, and builds a **realistic weekly schedule**. If time is not enough, it says so clearly and lists the tasks it could not place. If something unexpected happens, it fixes only the part of the schedule that is hit.

0.2 The main idea in one line

> *A vague wish ("plan my week for me") is turned into numbers, rules and steps that a computer can follow without guessing.*

This is what the course calls the **bridge between human and machine**. We build the bridge with the four pillars of Computational Thinking (CT).

0.3 Key decisions

#	Decision	Where
1	Target user: **one university student**, one **week** at a time (not the general public, not office workers).	§2.2
2	Problem model: **constrained optimization** — hard constraints must hold; soft preferences are improved.	§2.2
3	Decomposition by **questions the system must answer** (Problem Nature / Processing Logic / System Modules / Overload and Alerts), not by app screens or days of the week.	§3

#	Decision	Where
4	Group tasks by **behaviour** (Deep / Medium / Shallow), not by subject; use five patterns as reusable rules.	§4
5	Keep only the facts that **change a decision**: 8 task attributes (+1 optional) and a **672-slot** time grid of 15 minutes.	§5
6	One recipe of **5 steps** that combines deadline order, energy match, duration buffers and local repair.	§6
7	**AI is one part, not the whole system.** The language model reads text; a deterministic engine does all time arithmetic.	§7
8	**Feasibility before optimization**, **transparency before automation**, **stability before replanning**.	§8
9	Evaluate with a proposed 3-tier framework: 10 deterministic scenarios, an exploratory in-silico simulation (`SimulatedStudent`), and a proposed 1-week peer user study. All targets are **goals, not results**.	§9

0.4 How to read the numbers in this report

This is a proposal, so we have **no benchmark results yet**. To stay honest, every number is marked:

Tag	Meaning
(start value)	A number we chose to begin with, for example ×1.2 and ×1.35. We will test and change it.
(goal)	A target for evaluation, for example "at least 90% stability". It is not an achieved result.
(checked)	A number we re-computed by hand or with a small check script, for example 672 slots.

The test cases inside each pillar are **walk-throughs on paper** (we run the idea on Minh's week and see where it fails). They are not benchmark results.

1. Computational Thinking in this project

1.1 What "thinking" and "computational" mean here

- **Thinking** (technical view) is a problem-solving mindset: we frame a complex problem and organize the data logically, so that a human or a computer can carry out the solution. It happens in the human mind **before** any code is written.
- **Computational** describes the type of solution we look for. The solution must be structured, step by step, and precise enough that a computer (or a human acting like a machine) can run it **without guessing our intent**. "Thinking" is the human strategy. "Computational" is the target format of the solution.

A solution is computational when it is:

Characteristic	Meaning	How we check it in this report
Combinatorial and logical	Clear logic ("if X, do Y") and organized data, not vague feelings.	Every rule is an IF–THEN or a TRUE/FALSE check.
Deterministic and executable	Every step is defined. Ten people (or machines) get the same actions and the same result.	Fixed factors, fixed tables, fixed tie-break rules.
Scalable	It handles automation: it can serve many users, not only one.	Users only change the **data**; the model and the engine stay the same.

Each pillar section ends with a short **Computational check** using these three words, plus the **bridge between human and machine**.

1.2 The four pillars and what we do with each

Pillar	Meaning (course)	What we do in this project
1. Decomposition	Break a complex problem into smaller, more manageable parts. *Divide and conquer.*	Split "make a weekly schedule" into 4 perspectives and 4 modules.

Pillar	Meaning (course)	What we do in this project
2. Pattern Recognition	Look for similarities, trends or repetitions in the problem or in past experience. *Reuse solutions to similar problems.*	Find 5 repeating patterns in how students work, and reuse known scheduling methods.
3. Abstraction	Focus on the important information; filter out unnecessary details. *Keep only a universal core model.*	Keep 8 task attributes and a 672-slot grid; drop names, rooms, colours.
4. Algorithmic Design	Develop a step-by-step solution, rule book or recipe a machine can follow.	A 5-step recipe with clear tie-break rules, repair and overload handling.

1.3 What CT is *not* — and how we respect it

CT is **not** …	In our project
Thinking like a computer. Computers are literal and have no intuition. CT uses human creativity and critical thinking to find paths a machine can automate.	The student stays in control. The assistant gives recommendations; the student sets priorities, checks the breakdown and approves changes (§8.5).
Just programming. Programming is typing code. CT is the strategy **before** the code.	This proposal has no product code. We design first, then build.
Only for programmers.	Our users are students from any major. The core ideas (decompose, find patterns, keep what matters, write the steps) are the same for any person.

1.4 Design phase and action phase

CT is **only a strategy, not the final product**. The course separates two phases:

- **Thinking (design phase):** the four pillars. Purely mental, logical and structural.
- **Doing (action phase):** implementation and simulation. Real-world translation, constraints and testing with data.

This proposal is the **design phase**, plus a plan for the action phase (§10). Real systems are not built in a straight line. We follow the **continuous loop**: implement a draft → run an implementation or simulation → analyze the data to find new patterns → repeat to refine.

We also follow the course rule on **when** to start: **right from the problem definition**, not from algorithmic design. This is why §2 already uses decomposition, pattern recognition and abstraction.

2. Problem Definition and Context

2.1 Context and motivation

The problem starts with a simple feeling: *"I do not have enough time."* So a student makes a plan for the future. But most of the time the student fails to keep up with it. Why? A student estimates that homework takes 1 hour, but it lasts 2. One plan is missed, and the others fall along with it. The schedule only works for a machine, not for the student.

Students must balance study, jobs and daily life. Most use static to-do lists or basic calendars. The common symptoms are:

- the schedule needs repeated manual updates;
- tasks pile up near deadlines, which causes stress and burnout.

We did a **root cause analysis** and found three root causes:

Root cause	Meaning	Where we answer it
Root Cause 1 — Duration Bias (Planning Fallacy)	A systematic cognitive bias: people underestimate task completion times (we assume by 25–40%) and ignore the friction on the way.	Pattern 1: duration buffer (§4)
Root Cause 2 — Recovery Neglect	Time is treated as blocks, while biological energy rhythms and protected sleep are ignored.	Pattern 3: energy; 15-minute rest break (§4, §6)

Root cause	Meaning	Where we answer it
Root Cause 3 — Rigid Execution	A static timetable has no dynamic local repair. One unexpected delay makes the whole weekly plan collapse like dominoes.	Step 4: local repair (§6)

Note on evidence: the planning fallacy (people finish later than they predict) is well studied, for example by Buehler, Griffin and Ross (1994) with university students writing their theses. The exact sizes of our buffers are our own **start values** and must be tested (§4.4, §9).

2.2 From an ill-defined need to a well-defined problem

The first request was ill-defined: *"Help students manage their time."* It has no clear user, no definition of a good result and no limits. To get a clear and testable problem boundary, we reduced it in three discussions. In each, the team of 7 split into sub-groups and compared ideas.

Discussion 1 — Who is the target user? (Decomposition and Pattern Recognition)

Target user domain	Workload and schedule patterns	Autonomy and flexibility	Verdict and rationale
General public (sub-group 1)	Unstandardized; daily routines are highly unpredictable across professions.	High variability	**Rejected.** The domain is too broad to build testable rules.
Office workers (sub-group 2)	Fixed 8-hour company shifts; heavy external meeting demands.	Low autonomy	**Rejected.** Externally controlled calendars leave little room for AI scheduling.
Academic students (sub-group 3)	Recurring weekly lectures; fragmented free gaps; clear deadlines.	High flexibility	**Accepted.** A structured yet flexible domain, ideal for constraint-based scheduling.

→ **Decision:** narrow the project from "personal productivity" to **weekly academic scheduling for individual students**.

Discussion 2 — What information do we need? (Abstraction)

We want to keep only the necessary parameters, without making the user tired of entering data. The sub-groups compared three levels of abstraction:

Abstraction level	Data fields	Trade-off	Verdict and rationale
High-detailed raw input (sub-group 1)	Hourly energy level, homework texts.	High privacy concern; heavy computational data noise.	**Rejected.** Too much user data friction.
Minimal to-do list (sub-group 2)	Task title and deadline alone.	Not enough parameters for algorithmic placement.	**Rejected.** Cognitive load and splittability are missing.
Balanced feature set (synthesized from sub-groups 1 and 2)	Core commitments, task metadata, focus preferences.	Keeps the data that is necessary and keeps user friction low.	**Selected.** A sufficient and minimal model.

→ **Decision:** a balanced input set (§2.4).

Discussion 3 — What type of problem is it? (Pattern Recognition)

To find the mathematical form of the problem, the sub-groups compared three computational models:

Computational model	Mathematical mechanics	Trade-off	Verdict and rationale
Pure optimization (sub-group 1)	Maximize an objective function (productivity / comfort score).	May put work over fixed commitments or sleep if the score is high.	**Rejected.** It violates the non-negotiable sleep bounds.
Pure constraint satisfaction (sub-group 2)	Enforce TRUE/FALSE feasibility rules.	Makes valid schedules, but ignores energy rhythm and focus preferences.	**Rejected.** Feasible but rigid.
Constrained optimization (synthesized from sub-groups 1 and 2)	Enforce hard feasibility, then optimize focus periods and rest.	Good balance between feasibility and comfort.	**Selected hybrid model.**

→ **Synthesis: constrained optimization** with two tiers:

- **Tier 1 — Hard constraints (non-negotiable):** zero overlaps with fixed commitments (classes, work); protected sleep locking (at least 6 hours per night); schedule only inside the available free time; meet deadlines when feasible.
- **Tier 2 — Soft preferences (optimizable):** align the cognitive demand of a task with the student's focus periods (circadian rhythm); respect session lengths and rest breaks; minimize context-switching penalties; minimize schedule changes during repairs.

What does "when feasible" mean? A deadline is feasible when the capacity check of §6.5 does not prove an overload. If the check proves an overload, no schedule can meet all deadlines. The system then does **not** hide it: it keeps every other hard constraint, never places a task after its deadline, places what fits, and lists the rest (Step 5).

2.3 Formal problem statement

> **Given** a student's fixed commitments, academic tasks with estimated durations and deadlines, and personal scheduling preferences, **create** a realistic weekly schedule that respects fixed commitments and protected sleep, meets deadlines when feasible, and fits the preferences. **If** time is insufficient or constraints conflict, **transparently identify** the conflicts and the unscheduled tasks instead of claiming that an impossible schedule is feasible.

2.4 Inputs and outputs

	Content
Input 1 — Fixed commitments	Classes, work shifts, meetings, sleep, meals.
Input 2 — Task details	Title, estimated duration, deadline, priority (High / Medium / Low), whether the task can be split.
Input 3 — Preferences	Productive hours (for example "early morning" or "evening"), maximum focus length (for example Pomodoro 25 min or Deep Work 90 min), break needs.
Output 1 — Weekly schedule	What to do, when, and for how long (time blocks).
Output 2 — Rescheduling plan	After a disruption: adjust affected tasks, keep unaffected ones.
Output 3 — Overload warning	List of conflicts and tasks that do not fit.

2.5 Problem boundary

In scope

- A weekly schedule for **one** student.
- Academic deadlines and fixed commitments.
- Splitting large tasks into smaller sessions when allowed.
- Detecting conflicts and not enough time.
- Rescheduling after unexpected events or delays.
- Letting the student review and change the generated schedule.

Out of scope (non-goals)

- Changing sleep habits or giving health advice.
- Team calendars or several users.
- Doing academic work for the student.
- Guessing how hard a task is from its title alone.
- Guessing the student's physical or mental capacity.
- Long-term goals that last months or years.
- Managing every activity in the student's day.

These limits are on purpose. A wider scope adds uncertainty that makes the first version hard to build, test and improve. The first version puts **reliable weekly scheduling** before unlimited automation.

Assumptions and constraints

- The planning unit is one week. A deadline after the week is still considered (§5.6).
- The student enters the fixed commitments. The system cannot know about a missing class, so it warns that the schedule depends on this information.
- Duration estimates come from the student (§8.4).
- Time is measured in 15-minute slots (§5.6).

2.6 Representative use cases

ID	Use case	Expected behaviour	Test
UC1	Recurring classes or shifts	Reserve them every week as fixed commitments.	T01
UC2	Assignment details	Take title, duration, deadline, priority, splittable; ask if something essential is missing.	T01, T07, T08
UC3	Exam revision	Spread revision sessions over free time before the exam.	T02
UC4	Fragmented free gaps	Cut splittable tasks into sessions that fit the gaps.	T03
UC5	Protect sleep	Never place work in sleep time.	all tests (metric: hard-constraint violations)
UC6	Task splitting	Split only when allowed and never below the minimum part.	T03
UC7	Preferred focus slots	Prefer the student's focus periods without breaking a hard constraint.	T09
UC8	Event overlap	Move only the blocks hit by the new event.	T04
UC9	Task overruns	Update the remaining duration and re-plan.	T05
UC10	Overload warning	List unscheduled tasks and explain the conflict.	T06 (also T02)

The edge cases in §8.2 add more situations; the case "a deadline after the current week" has its own test (T10).

2.7 Success criteria (goals)

Situation	Success means
Feasible case	0 overlaps with commitments or sleep; all deadlines met.
Infeasible case	All unscheduled tasks and conflicts are reported clearly.
After a disruption	At least **90%** of unaffected blocks stay unchanged (goal).

Stability is measured as:

```
Schedule Stability = (Unchanged Unaffected Blocks ÷ All Previously Scheduled Unaffected Blocks) × 100%
```

The full list of metrics is in §9.4.

2.8 Running example: Minh's week

We use one student, **Minh**, in all four pillars.

Item	Data
Classes	Mon / Wed / Fri 13:00–17:00 · Tue / Thu 08:00–11:00
Other fixed items	Math quiz Wed 13:00 (during class) · Team meeting Thu 19:30–20:30
Sleep	23:30–06:30 (7 hours)
Meals	07:00–07:30 · 12:00–12:45 · 18:30–19:15
Preferences	"I work best early in the morning." · "I can focus for 90 minutes at most." (Deep Work 90)

Item	Data
To-do list (one sentence)	"Finish the final project (around 10 hours) by Sunday night, study for Wednesday's Math quiz (about 2 hours), write the lab report due Tuesday night (1 hour), and do the laundry and clean my room sometime this week."

By our count, Minh has **344 free 15-minute slots** in the week (about 86 hours) **(checked)**.

3. Pillar 1 — Decomposition

> *Break a complex problem into smaller, more manageable parts: divide and conquer.*

3.1 Where we start

The problem statement of §2.3 and Minh's week of §2.8. The problem is still one big block. Our lecturer asked for a **multilevel** decomposition, so we do not stop at one split: we break the problem into perspectives, and each perspective into smaller pieces, until each piece is small enough to compute.

3.2 The tough problems

1. "The AI does it" is a black box. If we cannot say *how* a task gets its place, we cannot test whether the result is right.
2. Real schedules break. A design that only builds a schedule cannot repair one.
3. If reading text, finding free time, placing tasks and updating the plan sit in one program, one change (for example a new AI provider) forces changes everywhere.

3.3 Options we tried

The team split into 3 sub-groups. Each sub-group proposed one way to break down the problem:

Attempt	Breakdown approach	Focus area	Trade-off	Verdict and rationale
1 (sub-group 1)	**By app screens**	Calendar, to-do, chat and notification screens	Focuses on how the user interface looks; fails to isolate the core scheduling logic.	**Rejected.** Black-box UI approach.
(Option, sub-group 2)	**By days of the week**	Monday to Sunday time blocks	Repeats the same scheduling logic on every day of the week.	**Rejected.** Repeated daily logic.
2 (sub-group 3, first version)	**By decision questions**: two perspectives, **What** + **How**	What is the problem made of? How do we solve each piece?	No plan for how to build it, and no branch for exceptions.	**Partial**
3 (sub-group 3, final version)	**By decision questions**: four perspectives, **Problem Nature** → **Processing Logic** → **System Modules** → **Overload & Alerts**	Problem-first decision perspectives	Exposes the decisions that the computational engine must solve.	**Chosen.** The 4-perspective model, a problem-first architecture.

Attempt 1 — Split by screens (rejected)

Works	Easy to share work in the team. Everyone can picture the product.
Fails	These are parts of the **product**, not parts of the **problem**. The hardest question — "how does the app decide where each task goes?" — stays inside one box called "the AI does it". It does not say what a *good* schedule is, so we cannot test it.
Test case	Minh types his sentence. Which screen decides that the 10-hour project must be split? Which one decides that the lab report (due Tuesday) comes before the project (due Sunday)? None. The calendar only shows the result; the chat box only receives the sentence.
Lesson	Decompose the **problem** first, and the **product** later. The pieces must explain *how* a decision is made, not only *where* it is shown.

Option — Split by days of the week (rejected)

Idea	One block per day, Monday to Sunday.
Fails	The scheduling decisions (which task first, which slot, what to do with a gap) are the same on every day. We would write and test the same logic seven times, and a task that can be split over several days belongs to no single day.
Lesson	A good piece of a problem is a different **question**, not a different **copy** of the same question.

Attempt 2 — Two perspectives: What + How (partial)

Works	We now know the ingredients of a schedule and the logic a human planner uses. Attempt 1's project problem is solved: the **How** perspective says "break big tasks into micro-tasks".
Fails	There is no plan for *how to build* it. Reading the sentence, finding free time, placing tasks and updating the plan are mixed in one program. If we change the AI provider for reading text (for example from OpenAI to Gemini), we must also touch the scheduling code.
Test case 1 (too much work)	Minh adds tasks until he has 15 hours of work but only 10 free hours before the deadline. The two perspectives assume everything fits, so there is no rule.
Test case 2 (emergency)	Tuesday 10:30: Minh writes "I have an urgent errand for 2 hours." The plan exists already. The two perspectives describe how to *build* a schedule, not how to *repair* one.
Lesson	Real schedules always break. We need a perspective that turns the logic into independent modules, and a perspective that breaks down the exceptions.

3.4 The chosen decomposition: four perspectives (Four-Perspective Problem Architecture)

Each perspective of the problem answers one core question. Each answer is split again until every piece is small enough to compute.

Perspective	Question	Pieces
Perspective 1: Problem Nature	What is the problem made of?	Free slots and task models; sleep bounds (at least 6 h); feasibility goals
Perspective 2: Processing Logic	How do we solve each piece?	Macro-to-micro task split; buffer scaling (×1.2 / ×1.35); multi-weight scoring
Perspective 3: System Modules	How do we build it?	M1 Natural Language Parser; M2 Grid Manager; M3 Math Solver Engine; M4 Local Repair Module
Perspective 4: Overload & Alerts	What if something goes wrong?	Workload > free time; transparent task listing; capacity warning alerts

Perspective 1 — Problem Nature: what is the problem made of?

Components. A schedule is a combination of three sets: the set of **free slots**, the set of **tasks**, and the set of **energy levels**.

Constraints. We separate two types:

Type	Meaning	Examples (Minh)
Hard constraint	Must never be broken.	No overlap with classes or other fixed commitments; sleep is protected (at least 6 hours per night); only free slots are used; the lab report ends before its deadline Tue 23:59 (**when feasible**, see §2.2 and §6.5).
Soft constraint	Flexible; used to rank choices.	Study Math in the morning; a short break after each hour of study.

What does the feasibility goal mean? Before placing anything, the system asks one yes/no question: is there enough free time before the deadlines? If yes, the system tries to meet every deadline (if its method still cannot place a part, it says "No schedule found", see §6.5). If no, the problem is **infeasible**. The system then says so clearly, and does not place a task after its deadline.

What is a good schedule? A good schedule is **not** the one that is packed full. In this order of importance:

1. It meets the hard constraints, and misses the fewest deadlines when not everything can fit (the missed tasks are listed, never hidden);

2. It matches the student's energy rhythm best;
3. It leaves room to breathe (rest breaks).

Perspective 2 — Processing Logic: how do we solve each piece?

We break down the thinking of a professional planner so we can teach it to a machine.

- **Break down tasks (macro to micro).** Split large macro-tasks into micro-tasks. The assistant *never blindly fragments a task on its own*: it prompts the student, and only decomposes if the student confirms and provides sufficient subtask details (§7.8.2); otherwise it preserves the task as an atomic block. Minh's "Final project, 10 h" becomes: Code feature A (2 h), Code feature B (2 h), Test and fix bugs (2 h), Write report (3 h), Make slides (1 h).
- **Scale the duration (buffer).** People underestimate time (Root Cause 1), so every estimate gets a buffer: ×1.2 for light and medium tasks, ×1.35 for heavy ones. Details in Pattern 1 (§4).
- **Score (multi-weight).** Decide which task goes first, and where, with four factors: **urgency** (how close the deadline is), **importance** (the impact of the task), **energy fit** (task difficulty against the energy of the slot) and a **context-switching penalty**. The proposed form is `Score = w1 × Urgency + w2 × Importance + w3 × Energy fit − w4 × Context-switching penalty`, where the weights are design parameters that we must calibrate in testing. Our recipe (§6.4) uses the same four factors, but applies them in a fixed order, so that two people (or machines) always get the same result.

Perspective 3 — System Modules: how do we build it?

We divide the system into four independent modules. Each one is easy to program, test and change.

Module	Input → Output	Example (Minh)
M1 — Natural Language Parser (language model)	The student's sentence → structured data in JSON (task name, deadline, duration, …)	The sentence → 5 task objects, e.g. `{"task_id": "LAB-1", "deadline": "Tue 23:59", "duration_min": 60}`
M2 — Constraint and Grid Manager	Calendar + hard constraints → time grid with free slots	Week minus classes, quiz, meeting, meals, sleep → Minh's free slots
M3 — Deterministic Engine (Math Solver) (core)	Micro-tasks + free slots → schedule	Checks hard constraints first, then puts each micro-task into a free slot using the recipe of §6
M4 — Dynamic Local Repair Module	New event or progress update → repaired schedule	Receives "urgent errand 2 h"; moves only what is hit

The modules talk only through these inputs and outputs. Changing the AI provider only changes **M1**. Changing the scheduling formula only changes **M3**.

Perspective 4 — Overload & Alerts: what if something goes wrong?

Schedules always break, so we branch the problem one case at a time. This way the assistant always has a backup plan.

Exception	Branch
Work is more than free time (for example 15 h of work, 10 h free)	Capacity warning alert. List the tasks that do not fit (transparent task listing). Suggest dropping or moving the lowest-priority tasks.
Emergency (for example a 2-hour interruption)	Keep everything before the current time; keep every block that is not hit; **move only the blocks the new event hits** to the nearest free slot, and widen the repair only if that is not possible.
Missing or invalid task information	Ask the student for the missing item (or use a clearly labelled default).

> **Why local repair, not a full rebuild?** An early idea was to send *all* unfinished tasks back to the scheduling engine after every change. We reject it because it moves blocks that were not affected, which confuses the student and lowers trust (§8.3, Example 4). Perspective 4 therefore uses local repair, and Step 4 of the recipe (§6.3) does the same.

3.5 The full decomposition tree (Four-Perspective Problem Architecture)

As presented in our architectural blueprint (Slide 27), the problem is broken down through a **multilevel hierarchy** structured into four distinct problem-solving perspectives, continuing until every leaf satisfies our formal stopping criteria.

1. Four-Perspective Problem Architecture (Slide 27 Alignment)

Perspective 1: Problem Nature	Perspective 2: Processing Logic	Perspective 3: System Modules	Perspective 4: Overload & Alerts
• **Free slots & task models** • **Sleep bounds** (≥ 6 h) • **Feasibility goals**	• **Macro to micro task split** • **Buffer scaling** ($\times 1.2 / \times 1.35$) • **Multi-weight scoring**	• **M1** NL Parser • **M2** Grid Manager • **M3** Math Solver Engine • **M4** Local Repair Module	• **Workload > Free time** • **Transparent task listing** • **Capacity warning alerts**

2. Atomic Leaves Stopping Rule (Formal Criteria)

We terminate decomposition along any branch when every leaf meets the **Atomic Leaves Stopping Rule**. A computational leaf must belong to one of three primitive forms:

#	Criterion	Formal Definition	Concrete System Example
1	**Atomic value**	A single primitive data value or scalar parameter	`duration_min = 120` , `priority = 5`
2	**Checkable Boolean rule**	A deterministic truth statement with binary evaluation	`ends_before_deadline == True` , `sleep_hours >= 6`
3	**Deterministic action**	A single, non-ambiguous computational operation	`place_task_in_slot(part, slot)` , `trigger_capacity_alert()`

3. Complete Multi-Level Decomposition Hierarchy

The hierarchy spans from the root problem (**L0**) down to individual computational actions (**L3**), fully resolving the "black box" of automated scheduling:

- **L0: Automated Weekly Academic Scheduler (Multi-Level Problem Hierarchy)**
  - **L1 — Perspective 1: Problem Nature** *(What is the schedule made of?)*
    - **L2 — Free slots and task models**
      - **L3 [Leaf — Atomic Value]:** Grid slots: `672` slots/week (15-min each) · Task attributes (8 required fields) · Energy levels: $\{1, 2, 3\}$.
    - **L2 — Biological sleep bounds**
      - **L3 [Leaf — Boolean Rule]:** Hard constraint: `sleep_hours >= 6` per night (`SLEEP` slots locked).
    - **L2 — Constraint taxonomy**
      - **L3 [Leaf — Boolean Rule]:** Hard rules: `no_overlap == True` , `no_sleep_used == True` , `ends_before_deadline == True` , `part_duration >= min_chunk_min` .
      - **L3 [Leaf — Boolean Rule]:** Soft rules: `match_energy(task, slot) == True` , `rest_break_follows_deep == True` .
    - **L2 — Feasibility goals**
      - **L3 [Leaf — Boolean Rule]:** Hierarchical check: `hard_constraints_satisfied == True` $\rightarrow$ `minimize_unplaced_tasks` $\rightarrow$ `maximize_energy_fit` .
  - **L1 — Perspective 2: Processing Logic** *(How do we systematically solve each piece?)*
    - **L2 — Macro to micro task decomposition**
      - **L3 [Leaf — Deterministic Action]:** Split large task into subtasks: `"Final project 10 h"` $\rightarrow$ Code A (2 h), Code B (2 h), Test (2 h), Report (3 h), Slides (1 h).
    - **L2 — Duration buffer scaling**
      - **L3 [Leaf — Deterministic Action]:** Compute `planned_duration = round_up_15(estimate * buffer_factor)` where factor is $\times 1.2$ (load 1–2) or $\times 1.35$ (load 3).
    - **L2 — Multi-weight priority scoring**
      - **L3 [Leaf — Atomic Value]:** Calculate `score = importance * urgency` where `importance = priority / 5` and `urgency = exp(-slack / 24)` .
  - **L1 — Perspective 3: System Modules** *(How is the software architecture decoupled?)*
    - **L2 — M1: Natural Language Parser (LLM)**
      - **L3 [Leaf — Deterministic Action]:** Ingest user sentence $\rightarrow$ validate and emit structured JSON task rows.
    - **L2 — M2: Constraint and Grid Manager**
      - **L3 [Leaf — Deterministic Action]:** Ingest calendar $\rightarrow$ lock fixed commitments and sleep $\rightarrow$ output free slot intervals.
    - **L2 — M3: Deterministic Engine (Math Solver)**
      - **L3 [Leaf — Deterministic Action]:** Sort tasks by score $\rightarrow$ greedy placement into best energy slots $\rightarrow$ emit schedule.
    - **L2 — M4: Dynamic Local Repair Module**
      - **L3 [Leaf — Deterministic Action]:** On disruption $\rightarrow$ freeze unaffected blocks $\rightarrow$ relocate impacted blocks to nearest valid slot.

- **L1 — Perspective 4: Overload & Alerts** *(What happens when limits are breached?)*
  - **L2 — Capacity overload detection**
    - **L3 [Leaf — Boolean Rule]:** Check `sum(planned_work) <= free_time_before_deadline` . If false, trigger capacity warning alert.
  - **L2 — Transparent task listing**
    - **L3 [Leaf — Deterministic Action]:** Output unscheduled tasks list + exact shortage in minutes (never hide infeasibility).
  - **L2 — Real-time emergency handling**
    - **L3 [Leaf — Deterministic Action]:** Freeze historical and unaffected blocks; move only the conflicting blocks (Step 4 repair).
  - **L2 — Incomplete task data recovery**
    - **L3 [Leaf — Deterministic Action]:** Detect missing essential attributes → prompt student with clarification dialog.

3.6 Re-checking the failing test cases

Check	Result for Attempt 3
Fixes Attempt 1?	Yes. Perspective 2 says the project must be broken into micro-tasks, and the priority score puts the lab report before the project (§6.4).
Fixes Attempt 2, test case 1?	Yes. Perspective 4 has a branch for "work > free time".
Fixes Attempt 2, test case 2?	Yes. Perspective 4 has a branch for emergencies, and M4 handles it.
Are the parts independent?	Yes. Each module has its own input and output, so the team can build and test them separately.

3.7 What decomposition cannot solve

Decomposition shows **which pieces** are needed, but not **their values**. How long does a "2-hour" task really take? When does Minh have high energy? Which tasks should be grouped? To answer these we must look at how people usually behave. That is Pillar 2.

3.8 Computational check

Characteristic	How Pillar 1 meets it
Combinatorial and logical	Every perspective answers a fixed question (Problem Nature: what; Processing Logic: how; System Modules: how to build; Overload & Alerts: what if). Every leaf is a value, a TRUE/FALSE rule or one action. Every exception is an IF–THEN branch (IF work > free time THEN alert and suggest).
Deterministic and executable	Each module has a defined input and output. The pieces are the same for everyone who uses the perspectives. *Note:* M1 reads free text with a language model, so M1's output is shown to the student for confirmation. From the structured task rows onward, the same input always gives the same result.
Scalable	The 4 perspectives and 4 modules do not depend on one student. Another student only brings different data.
Bridge between human and machine	A vague wish — "plan my week for me" — becomes 4 perspectives, 4 modules and concrete micro-tasks that a program can process.

Hand-off to Pillar 2: we now have the pieces (tasks, free slots, energy levels, constraints) and the modules. Next we look for patterns in how people really work, so we can fill these pieces with realistic values.

4. Pillar 2 — Pattern Recognition

> *Look for similarities, trends or repetitions in the problem or in past experience, and reuse the solution for similar problems.*

4.1 Where we start (output of Pillar 1)

Decomposition gave us the pieces: tasks, free slots, energy levels, hard and soft constraints, and 4 modules. But the pieces are still **empty or unreliable**. Minh says the lab report takes "1 hour" — does it really? When is his energy high? Should the laundry go between two study sessions?

Goal of this pillar: find the common patterns (similar behaviour that repeats across people, days and tasks). Each pattern becomes **one reusable rule**, so the schedule is realistic and not only perfect on paper.

4.2 The tough problems

1. Time estimates are wrong, and wrong in the same direction again and again.
2. Not all tasks need the same kind of time (a long quiet block vs. a few spare minutes).
3. Energy changes during the day, so the *same* free hour is not equal for every task.
4. Switching between types of work, and tiny gaps, waste time.

4.3 Two kinds of patterns we look for

Kind	Meaning	What we reuse
A. Patterns in how people work (from past experience)	Behaviour that repeats across people, days and tasks.	Five reusable rules (§4.5)
B. Patterns in the problem itself	Our problem looks like problems that are already solved.	Known methods: Earliest Deadline First, greedy placement, urgency × importance, repair instead of rebuild (§4.6)

We also decided **how to classify tasks**. Two sub-groups debated the classification scheme:

Classification scheme	Category scheme	Flaws and limitations	Verdict and rationale
By course subject (sub-group 1)	Math, Physics, Literature, Electives	A subject title does not show the duration, the difficulty or the best focus time.	**Rejected.** No scheduling value.
By cognitive behavioral workload (sub-group 2)	**Deep Work** vs. **Medium Work** vs. **Shallow Work**	Needs a first guess of the type for each task (the student can change it, see Pattern 2).	**Selected.** Directly computable: it maps to time-block length, energy needs and context-switching cost.

4.4 Options we tried

Attempt	Idea	Verdict
1	Trust the user's numbers; treat every task the same.	**Rejected**
2	One global rule: add 20% to every estimate.	**Partial**
3	**Five patterns**, each with a detection rule and a reusable action.	**Chosen**

Attempt 1 — Trust the user exactly (rejected)

Idea	Use the durations Minh typed (lab report 1 h, Math 2 h, project 10 h) and place tasks wherever there is free time.
Works	Simple. The user feels in control.
Test case 1	Monday morning: Math is planned 07:30–09:30 and project coding 09:30–11:30. Math really takes 2 h 40 min (about 33% longer), so coding starts 40 minutes late and runs into lunch. Every later block on Monday slides, because the plan has no slack.
Test case 2	Math study is placed at 22:00, the only free 2 hours left that day. Minh is tired, gives up after 30 minutes, and the task is marked "done" on paper only.
Lesson	People are not machines. They make the same mistakes again and again, and their energy follows the same daily curve. These repetitions are the patterns we should find and use.

Attempt 2 — One global rule: add 20% (partial)

Idea	The best-known pattern is the **planning fallacy**: people underestimate how long tasks take. So the system adds a **duration buffer**: every duration × 1.2.

Works	Most overruns of Attempt 1 now fit in the plan. It is one simple rule for every user.
Fails	The same 20% is added to a 30-minute laundry task and to a 3-hour report. Our own assumption is that homework is underestimated by 25–40%, so hard tasks still overrun.
Test case	The rule changes only *how long*, never *when* or *with what*. Math (hard thinking) can still be placed late at night. The laundry is placed between two study blocks, so Minh switches from study to chores and back. The 15-minute gap between lunch (ends 12:45) and class (13:00) stays empty every Monday, Wednesday and Friday, because no task fits.
Lesson	One pattern is not enough. We need a **set** of patterns, each about a different question: how long does it really take, what type of work is it, when can the person do it, what should be grouped, and what to do with tiny gaps.

4.5 The chosen solution: five patterns

For every pattern we write down three things: **what repeats** (the similarity), **how the system recognizes it** (a rule a machine can check), and the **reusable action** (the same solution for every task or slot that matches).

> All factors and thresholds below are **start values**. They come from our assumptions and general findings about behaviour. We will test and change them with data (§4.8, §9).

The five patterns in one table (the details of each pattern follow):

Behavioral pattern	Real-world human tendency	Deterministic logic rule (proposal)
Pattern 1: Planning Fallacy	Humans systematically underestimate task completion times (a 25–40% duration bias, assumed as an initial heuristic to be tested).	Buffer scaling: × 1.2 for light and medium tasks; × 1.35 for heavy academic tasks.
Pattern 2: Workload Types	Tasks need different focus levels (Deep, Medium, Shallow Work).	Match the task cognitive load (1–3) to the focus duration; cap Deep Work at 90 min per session.
Pattern 3: Circadian Rhythms	Daily focus peaks in the morning or evening and dips after lunch (the student can adjust it).	Put Deep Work (load 3) in peak-energy slots (level 3); put chores in low-energy slots.
Pattern 4: Context Switching	Switching between different subjects or types of work costs focus (10–15 min, an initial heuristic).	Batch similar tasks together; add a penalty when different task types are back to back.
Pattern 5: Dead-Zones	Free gaps shorter than 30 min between fixed events are easily wasted.	Never schedule Deep Work in a gap shorter than 30 min; use it for Shallow work or mark it as active rest.

Pattern 1 — Planning Fallacy *(duration bias)*

Aspect	Details
What repeats	People estimate homework shorter than it is. Our assumption, to be tested, is a duration bias of 25–40%. They also tend to put off a task until close to its deadline.
How it is recognized	Every task that is not FIXED. Its `cognitive_load` (Pillar 3) tells us whether it is light, medium or heavy.
Reusable action	**Planned duration = estimate × 1.2** for light or medium tasks (load 1–2), **× 1.35** for heavy academic tasks (load 3), then **rounded up to the next 15 minutes**. The buffer is applied **once**. The engine prefers the earliest suitable time, so work is not left to the last minute.
Minh	Lab report 60 → 72 → **75 min** · Math 120 → 162 → **165 min** · Laundry 45 → 54 → **60 min**

> *Why are the factors lower than 25–40%?* The factors ×1.2 and ×1.35 are start values, not proof. The rounding up to 15 minutes adds some extra time, and a buffer that is too large would waste free time. This is exactly what we want to test (§4.8, §9.6).

Pattern 2 — Workload Types *(Deep, Medium and Shallow Work: what kind of time a task needs)*

Aspect	Details
What repeats	Tasks fall into three groups. **Deep Work** needs a long, continuous block (1–2 h) of full focus. **Medium Work** needs some focus, but can be interrupted. **Shallow Work** is small and light (15–30 min).

Aspect	Details
How it is recognized	M1 reads the action word of the task. A keyword table gives a **first suggestion**: study, code, write, design, solve → **Deep** (load 3) · review, edit, read → **Medium** (load 2) · laundry, clean, email, buy, pay → **Shallow** (load 1). If the word is unknown, the assistant **asks the student once** and remembers the answer. The student can always change the suggestion.
Reusable action	Match the `cognitive_load` (1–3) of the task to the length of its block. A Deep task gets one continuous block, **capped at 90 minutes per session** (or the student's own maximum focus, if shorter; Minh: 90 min). Shallow tasks can use short gaps.
Minh	Math, project coding, project report → Deep · lab report → Medium · laundry, cleaning → Shallow

> *Why a suggestion only?* A title cannot reliably show how hard a task is — this is one of our non-goals (§2.5). Minh's sentence says "**write** the lab report", so the table suggests Deep (load 3). Minh changes it to Medium, because the lab report only needs him to put in results he already has. This is the human-in-the-loop idea of §8.5. The keyword table is used **only for the load**, never for importance (§8.3, Example 1).

Pattern 3 — Circadian Rhythms *(recovery neglect: energy and the daily rhythm)*

Aspect	Details
What repeats	Focus peaks in the morning or in the evening, and dips after lunch. For most people it is highest around 9:00–12:00 and falls sharply in the early afternoon. Hard homework placed late at night is often abandoned, while light chores are still finished well at low-energy times.
How it is recognized	Every 15-minute slot gets an `energy_level` (1–3) from a **default daily curve**. The student can adjust it: the productive hours they enter ("early morning" or "evening") **replace** the default.
Reusable action	Put Deep Work (load 3) in peak-energy slots (level 3) and chores in low-energy slots. The engine matches each task's `cognitive_load` to the slot's `energy_level`.
Minh	Says "early morning", so 06:30–12:00 → 3 · 12:00–15:00 → 1 · 15:00–21:00 → 2 · 21:00–23:30 → 1

Pattern 4 — Context Switching

Aspect	Details
What repeats	Switching between different subjects or kinds of work (for example schoolwork and housework) costs time to regain focus (our assumption: 10–15 minutes each time).
How it is recognized	Two neighbouring tasks of different types (for example Deep ↔ Shallow).
Reusable action	Batch similar tasks together (for example housework in one session). Add a penalty when different task types are placed back to back. When two places are equally good, prefer the slot next to a task of the **same** type.
Minh	Laundry and room cleaning are placed back to back in one chores session.

Pattern 5 — Dead-Zones

Aspect	Details
What repeats	Free gaps shorter than 30 minutes between fixed events are easily wasted, because people do not want to start a big task in them.
How it is recognized	A run of FREE slots shorter than 30 minutes with a non-free slot on both sides.
Reusable action	Never schedule Deep Work there. Use the gap for Shallow work, or mark it as **active rest** (stretching, a walk, water), so it becomes a planned break and not lost time. In our recipe a dead zone is marked as active rest first; very small Shallow tasks go to the end-of-day shallow block (§7.2).
Minh	12:45–13:00 on Mon / Wed / Fri, and 19:15–19:30 on Thursday (between dinner and the team meeting; found when we built the grid) **(checked)** → active rest.

4.6 Patterns in the problem itself (reuse known solutions)

When we look at the problem, not at the people, it looks like problems that are already solved. We reuse their ideas instead of starting from nothing.

Sub-problem	Similar known problem	Idea we reuse
Which task goes first?	Scheduling jobs with deadlines	**Earliest Deadline First**, combined with importance (**urgency × importance**)
Where does a task go?	Filling free intervals	**Greedy placement** into suitable free intervals
Which plans are allowed?	Constraint satisfaction	Hard rules as TRUE/FALSE checks; soft rules as ranking
A plan breaks	Repairing a plan	**Local repair** first, widen only when needed

The details are in Pillars 3 and 4.

4.7 Re-checking the failing test cases

Test case	Result with Attempt 3
Math overruns (Attempt 1)	Planned as 120 × 1.35 = 162 → **165 min** on the 15-minute grid, so a real 2 h 40 min (160 min) fits inside the plan.
Math at 22:00 (Attempts 1, 2)	Math is Deep (load 3) and 22:00 has energy 1, so it no longer matches. Math moves to the morning.
Same 20% for everything (Attempt 2)	Hard tasks × 1.35, light tasks × 1.2.
Laundry between study blocks (Attempt 2)	Chores are batched into one session.
Empty 15-minute gaps (Attempt 2)	Marked as active rest, so they become planned breaks.

4.8 How we will check and refine the patterns

The patterns are our best first guess. We will compare **estimated vs. actual duration**, **number of reschedules**, **unused gaps** and **user overrides** (for example, how often students change a Deep suggestion). If the data disagrees, we change the factor or the rule. A walk-through already showed a limit of the buffers: a task that overruns by +50% still breaks the plan (we test +25% and +50% in §9.6).

4.9 Computational check

Characteristic	How Pillar 2 meets it
Combinatorial and logical	Every pattern is an IF–THEN rule: IF load = 3 THEN × 1.35; IF gap < 30 min THEN active rest; IF a neighbour has the same type THEN prefer it.
Deterministic and executable	Fixed factors (1.2 / 1.35), a fixed keyword table and a fixed energy curve: the same task always gets the same planned duration, type and best time (unless the student overrides).
Scalable	The patterns come from how people in general behave, so they work for a new student from day one. Personal preferences only change the input values.
Bridge between human and machine	"People are bad at estimating" becomes × 1.35. "I'm a morning person" becomes energy 3 from 06:30 to 12:00.

Hand-off to Pillar 3: the patterns tell us which properties matter: duration (with buffer), type of work, energy, and whether a task has a deadline. Next we strip away everything else and build the data model.

5. Pillar 3 — Abstraction

> *Focus only on the important information and filter out unnecessary details. Strip out noise and keep one universal core model.*

5.1 Where we start (output of Pillars 1 and 2)

The input is still human: a sentence such as "study for Wednesday's Math quiz (about 2 hours)", and calendar events full of titles, rooms, colours and notes. Pattern Recognition told us what matters: duration (with buffer), type of work, energy, deadlines.

Goal of this pillar: for the computer to calculate, remove the details it does not need (the exact name of the task, what the homework is about) and keep only the **mathematical properties** of the problem: one data model for tasks and one for time.

5.2 The tough problems

1. A computer needs numbers and categories, not sentences ("about 2 hours", "sometime this week").
2. We must not keep too much (noise) or too little (missing decisions).
3. Time itself must be turned into something a computer can check.

5.3 Options we tried

Attempt	Idea	Verdict
1	Keep the raw input as it is.	**Rejected**
2	Keep only name, duration and deadline.	**Partial**
3	Keep exactly what **changes a decision**: 8 task attributes + a 672-slot grid.	**Chosen**

Attempt 1 — Keep the raw input (rejected)

Idea	Store Minh's sentence as text and copy each Google Calendar event with all fields: title, description, room, attendees, colour, notes, start, end. Nothing is lost.
Works	Easy to sync with apps the student already uses.
Fails	Most of these fields never change where a task goes. A teacher's name or a colour cannot move a task.
Test case	To compare "about 2 hours" with a free gap, the machine needs a number. "Wednesday" means a different date every week, and "sometime this week" (the chores) has no date at all. Also, a calendar event needs a start and end time, but choosing the start time is exactly what we want the app to do. This model describes the **answer**, not the **problem**.
Lesson	More detail is not better. The computer needs numbers and categories, not sentences.

Attempt 2 — Keep only name, duration and deadline (partial)

Idea	The other extreme: every task = `{name, duration, deadline}` . Time is only "free" or "busy".
Works	Everything is computable: we can compare a duration with a free gap.
Test case 1	Minh's project (10 h) is one task of 600 minutes. His biggest free gap is under 6 hours (the longest is 5 h 45 min on Saturday afternoon), so the model says "impossible", even though he has about 86 free hours in the week. The model does not know the task can be split.
Test case 2	Math study and laundry look the same to the model. The laundry can take Monday's 07:30 peak hour while Math goes to 21:00. There is no type of work, so Pattern 3 (energy) cannot be used.
Test case 3	The lab report (Medium) and the project (High) cannot be compared, because there is no priority. The chores have no deadline, so they cannot even be sorted. Fixed classes and movable tasks are mixed in one list.
Lesson	Abstraction does **not** mean "remove as much as possible". It means **remove what does not change the decision, and keep everything that does**. This gives us a test for every fact.

5.4 The chosen solution

For every fact we ask: **"If this value changes, can the place of the task in the schedule change?"** If no, we drop it. If yes, we keep it as a number, a true/false value or a category.

Step 1 — Apply the decision test

Fact	Can it change the schedule?	Keep?	Becomes
Task name ("Code feature A")	No. It is only shown to the student.	Drop	`task_id` (a label; the algorithm never reads the name)

Fact	Can it change the schedule?	Keep?	Becomes
Content of the homework, subject, teacher, room, colour, notes	No.	Drop	—
Kind of item: fixed / has a deadline / habit	Yes. Fixed items never move; habits have no deadline.	Keep	`category`
Estimated time	Yes. Longer tasks need bigger gaps.	Keep	`duration_min`
Due time	Yes.	Keep	`deadline`
High / Medium / Low	Yes. It decides what goes first.	Keep	`priority`
Can it be done in parts?	Yes. It decides whether a 10 h task can use 1 h gaps.	Keep	`is_splittable`
Smallest useful part	Yes. A 15-minute piece of coding is useless.	Keep	`min_chunk_min`
Deep / Medium / Shallow (Pattern 2)	Yes. It decides the best time of day.	Keep	`cognitive_load`
Must it come after another task? (found when we checked Minh's project: the test must come after the code)	Yes.	Keep	`depends_on` (optional)

Step 2 — The task model

Attribute	Data type	Description
`task_id`	String	Unique identifier of the task
`category`	Enum	FIXED, DEADLINE, HABIT
`duration_min`	Integer	Estimated duration, a multiple of 15 minutes
`deadline`	Time point	Time the task must be finished (empty for housework)
`priority`	Integer (1 to 5)	Importance. High = 5, Medium = 3, Low = 1. Chosen or confirmed by the student; the assistant may suggest (§8.3).
`is_splittable`	Boolean	Whether the task may be cut into smaller parts
`min_chunk_min`	Integer	Minimum length of one part (for example 45 minutes)
`cognitive_load`	Integer (1 to 3)	3 = high focus (Deep), 2 = medium, 1 = manual or administrative (Shallow)
`depends_on` *(optional)*	List of `task_id`	Tasks that must be finished first. Empty for most tasks.

The first 8 attributes were the result of the decision test. `depends_on` was **added later**, when we walked Minh's project through the model and saw that "Test and fix bugs" could be placed before the code. This is one small example of the continuous loop (§1.4).

`planned duration` is **not** an input. It is computed in the recipe: `estimate × buffer`, rounded up to 15 minutes (Pattern 1). The scheduler uses the planned duration.

The model has **one more small part: the student's profile** (not a task): sleep window and minimum sleep (hard lock, default 6 hours), productive hours, maximum focus length, and the buffer factors.

Minh's tasks after abstraction

task_id	category	duration_min	deadline	priority	is_splittable	min_chunk_min	cognitive_load	depends_on
MATH-1	DEADLINE	120	Wed 13:00	5	True	45	3	—
LAB-1	DEADLINE	60	Tue 23:59	3	False	—	2	—

task_id	category	duration_min	deadline	priority	is_splittable	min_chunk_min	cognitive_load	depends_on
PRJ-1 (code A)	DEADLINE	120	Sun 23:59	5	True	45	3	—
PRJ-2 (code B)	DEADLINE	120	Sun 23:59	5	True	45	3	—
PRJ-3 (test)	DEADLINE	120	Sun 23:59	5	True	45	2	PRJ-1, PRJ-2
PRJ-4 (report)	DEADLINE	180	Sun 23:59	5	True	45	3	—
PRJ-5 (slides)	DEADLINE	60	Sun 23:59	5	False	—	2	—
LAUNDRY	HABIT	45	(empty)	1	False	—	1	—
CLEAN	HABIT	30	(empty)	1	False	—	1	—

FIXED items (for example `CLASS-MON`, 240 min) keep their calendar time only long enough to be written into the time grid. After that they are just OCCUPIED slots.

Before and after — what M1 produces

- Human view: *"…study for Wednesday's Math quiz (about 2 hours)…"*
- Machine view:

{"task_id": "MATH-1", "category": "DEADLINE", "duration_min": 120, "deadline": "Wed 13:00", "priority": 5, "is_splittable": true, "min_chunk_mi

Because the language model can make mistakes, the student **confirms** this machine view before it is used.

5.5 The time model

The student's week becomes an array of **672 time slots** of 15 minutes (4 slots/hour × 24 hours × 7 days). Each slot holds two values: a **state** and an **energy_level** (1 to 3, from Pattern 3).

- States from the input: `FREE` (confirmed available), `OCCUPIED`, `SLEEP`, `UNKNOWN` (availability not confirmed yet; the recipe does not use it, §8.5).
- States set later by the recipe: `BLOCK` (a placed task), `BREAK` (a 15-minute rest break), `REST` (a dead zone used as active rest).

Why 15 minutes?

Slot size	Slots per week	Effect	Decision
1 minute	10,080	Very precise, but 15× more work, and nobody needs to start at 09:07	Too fine
60 minutes	168	A 45-minute task wastes 15 minutes; short gaps between classes disappear	Too coarse
15 minutes	**672**	Matches Pomodoro and Deep Work lengths (25 → 30, 45, 60, 90 min); waste is at most 14 minutes	**Chosen**

Why a separate SLEEP state? Sleep is not a normal occupied block. The algorithm **never uses SLEEP slots**. Protecting sleep is a hard constraint, and changing sleep habits is a non-goal (§2.5), so the system also never suggests cutting sleep.

Minh's Monday as the computer sees it

Time	State	energy_level	Reason
00:00–06:30	SLEEP	—	sleep
06:30–07:00	FREE	3	early morning (Minh's preference)
07:00–07:30	OCCUPIED	—	breakfast
07:30–12:00	FREE	3	morning peak
12:00–12:45	OCCUPIED	—	lunch
12:45–13:00	FREE → REST	1	dead zone (Pattern 5)
13:00–17:00	OCCUPIED	—	class
17:00–18:30	FREE	2	afternoon
18:30–19:15	OCCUPIED	—	dinner

Time	State	energy_level	Reason
19:15–21:00	FREE	2	evening
21:00–23:30	FREE	1	late evening
23:30–24:00	SLEEP	—	sleep

Counts for Minh's week (checked): 672 slots = 196 SLEEP + 132 OCCUPIED + **344 FREE** (no UNKNOWN slots, because Minh confirmed his free time). After the 4 dead zones are marked REST, **340** slots stay FREE.

Deadline after the planning week. A task due next week may need work this week (edge case 9). The grid has 672 slots for one week. When a deadline falls after Sunday, we add one more 672-slot week to the model and show only the current week to the student *(proposed rule, to test with T10)*.

5.6 Constraints become rules

Hard constraints → checks that must be TRUE	Soft constraints → used to rank good / better / best
H1. A block uses only FREE slots.	S1. `cognitive_load` matches the slot's `energy_level` (Pattern 3).
H2. A part is a run of consecutive slots; it is never shorter than `min_chunk_min` (and never longer than the maximum focus).	S2. A 15-minute rest break follows Deep Work (burnout prevention, §7.4).
H3. A block ends before its deadline (when feasible). If no valid slot exists before the deadline, the task is never placed late: it goes to the unplaced list (Step 5).	S3. Tasks of the same type sit next to each other (Pattern 4).
H4. A task starts after the tasks in its `depends_on`.	S4. Earlier is better than later (Pattern 1, procrastination).
H5. SLEEP slots are never used.	

5.7 The everyday problem becomes a computational problem

The abstract problem: for every task (or every part of a splittable task), find a run of `planned duration ÷ 15` consecutive FREE slots that ends before the deadline, so that all hard rules are TRUE and the match between `cognitive_load` and `energy_level` is as good as possible.

This is a classic **constraint satisfaction / interval scheduling** problem. Because it now looks like a known type of problem, Pillar 4 can reuse known ideas (sorting by priority, greedy placement, Earliest Deadline First) and does not start from nothing.

How big is it? Minh's week has **15 parts** to place and **340 free slots** *(checked, with the planned durations of §6)*. If we tried every start position for every part, there would be up to about $340^{15} \approx 10^{38}$ combinations (an upper bound before the rules remove some). Trying everything is far too slow. We need a step-by-step procedure.

5.8 Re-checking the failing test cases

Test case	How the model handles it
Text the machine cannot compare (Attempt 1)	`duration_min` = 120, `deadline` = Wed 13:00
Model needed a start time (Attempt 1)	The start slot is now the **unknown** the algorithm solves for
10 h project "impossible" (Attempt 2)	`is_splittable` = True and `min_chunk_min` = 45 → it can use smaller gaps
Laundry in the morning peak (Attempt 2)	`cognitive_load` 1 vs. 3, and an `energy_level` on every slot
No priority; chores without a deadline (Attempt 2)	`priority` 1–5; category HABIT with an empty deadline
Test placed before the code	`depends_on`

5.9 Computational check

Characteristic	How Pillar 3 meets it
Combinatorial and logical	Every fact went through the same yes/no decision test. Every constraint is either a TRUE/FALSE check or a ranking rule.
Deterministic and executable	No free text is left. Each attribute has a fixed data type (String, Enum, Integer, Boolean), so two people modelling the same task get the same row.
Scalable	One universal model: any student only changes the data. 672 slots per week is a small, fixed size for any computer.
Bridge between human and machine	"Study for Wednesday's Math quiz, about 2 hours" becomes `{MATH-1, DEADLINE, 120, Wed 13:00, 5, True, 45, 3}`.

Hand-off to Pillar 4: input = the task rows and the 672-slot grid. Output needed = a slot range for every task (or part), a list of tasks that do not fit, and a way to redo it when the week changes.

6. Pillar 4 — Algorithmic Design

> *Develop a step-by-step solution, rule book or recipe to solve the problem. Write down complete steps that a machine can follow.*

Objective: build a clear, step-by-step recipe so that a computer can generate a realistic schedule and adapt to disruptions without manual rescheduling.

6.1 The tough problems

1. Which task goes first, and where does each one go? Trying everything is far too slow (§5.7).
2. A recipe that follows only deadlines puts hard work at bad hours; a recipe that follows only energy can miss deadlines.
3. A plan with no slack breaks at the first overrun, and a plan that is rebuilt after every change confuses the student.
4. Sometimes there is simply not enough time. The recipe must say so, and not pretend.

6.2 Options we had

The team split into **3 sub-groups working in parallel**. Each proposed a strategy. We also list other options we considered.

Option	Idea	Strength	Limit	Decision
A. Try everything (brute force)	Test every placement and keep the best.	Finds the best answer.	About 10^{38} placements for one week (§5.7).	Rejected
B. Ask a language model to make the schedule ("pure LLM prompting")	Give the model the tasks and calendar.	Easy to start.	Poor at time arithmetic; can invent overlaps and break sleep.	Rejected (§7.1)
S1. Deadline and urgency (sub-group 1)	Rank tasks by urgency and priority; place the most urgent first into the earliest free slot.	Fast and clear; urgent work is considered first.	Ignores energy and estimate errors; can place Deep Work late at night (for example Math at 22:00). It is not guaranteed to find an on-time plan when tasks cannot be split and free time is full of gaps (like packing boxes).	**Used** for task order
S2. Cognitive load and energy (sub-group 2)	Match task difficulty to energy; add breaks.	Protects focus and prevents burnout.	If peak slots are used *strictly*, they run out in deadline-heavy weeks, and tasks pile up.	**Used** as a **soft** rule, not a strict one
S3. Buffers and local repair (sub-group 3)	Add extra time to each task; after a disruption, lock the past and move only the task that is hit.	Stable: aims to keep at least 90% of the plan (a goal, not a result).	It cannot build the first schedule. It needs an engine to start.	**Used** as Step 2 and Step 4

Option	Idea	Strength	Limit	Decision
C. Exact solver	Find the best plan with a constraint solver.	Can *prove* that no plan exists.	Heavier; harder to explain to the student; soft goals need weights.	Used as a **reference check** in tests (§9.2), not in the product

Decision: combine **urgency + suitable time slots + buffer and repair rules**. No single strategy is enough.

6.3 The recipe: five steps

We combined the deadline discipline of S1, the energy awareness of S2, and the buffers and repair logic of S3.

Step	Name	What the machine does
1	**Lock Fixed Commitments and Protect Biological Sleep**	Mark classes, shifts, meals and meetings as OCCUPIED. Mark sleep as SLEEP. Set the energy level of every slot. Mark dead zones (FREE gaps shorter than 30 min) as REST.
2	**Task Preprocessing** (buffer scaling and splitting)	Planned duration = estimate × buffer, rounded up to 15 min (once). Split large tasks into parts of 45–90 minutes (not shorter than `min_chunk_min`, not longer than the maximum focus). Compute the priority score and sort.
3	**Priority and Circadian Energy Placement**	Check capacity first. Then place parts in order: urgent first; demanding work in high-energy slots; light chores in low-energy slots; a 15-minute rest break after Deep Work; chores batched into one session.
4	**Minimal Rescheduling and Local Repair**	When a new event arrives: freeze the rest of the week and move only the blocks that are hit to the nearest free slot that still meets the hard constraints. If this fails, widen the repair step by step.
5	**Deterministic Overload Reporting**	If time runs out: place the most critical tasks first, list every unplaced task, and suggest actions (extend a deadline, drop or postpone a non-essential task).

How the recipe connects to the earlier pillars:

Recipe step	Module (Pillar 1)	Patterns (Pillar 2)	Model (Pillar 3)
1	M2	P3, P5	slot states, energy_level
2	M1 + M3	P1, P2	`duration_min`, `cognitive_load`, `is_splittable`, `min_chunk_min`
3	M3	P2, P3, P4, P5	`category`, `deadline`, `priority`, `depends_on`
4	M4	— (local repair)	FREE / BLOCK slots
5	M3 + M4	—	unplaced list

6.4 Details that make the recipe executable

The course says a computational solution must leave **no room for ambiguity**: ten people (or machines) must get the same result. So every choice needs a clear rule, including ties.

Step 2 — priority score

```
slack_hours = (hours from now until the deadline) − (planned duration in hours)
urgency     = exp( −max(slack_hours, 0) ÷ 24 )      # 1 when no slack; about 0.37 with 24 h; near 0 when far
importance  = priority ÷ 5
score       = importance × urgency                   # tasks with no deadline: urgency = 0
```

The urgency grows sharply as the deadline comes close (the "exponential surge"). The value 24 hours is a **start value**.

Order = score (high first) → earlier deadline → order in the student's list. A task is placed only after the tasks in its `depends_on`.

Step 2 — splitting. For a splittable task, use the smallest number of parts that keeps every part within the maximum focus (Minh: 90 min), and make the parts as equal as possible, in multiples of 15 minutes. If a part would be shorter than `min_chunk_min`, use fewer parts. An unsplittable task is one part.

Human-in-the-loop rule: The system prompts the student before splitting; it decomposes only when the student confirms and provides sufficient subtask details (§7.8.2). If the student declines, it keeps the task as a single block and alerts if no continuous slot fits.

Step 3 — choosing a slot. A start position is allowed only if all hard rules hold (H1–H5). Among allowed positions, pick the one with the smallest value of, in this order:

1. **energy mismatch** = average of | `cognitive_load − energy_level` | over the slots of the part;
2. **neighbour score** = −1 for each neighbouring block of the same load, +1 for each neighbouring block that is Deep next to Shallow (or the reverse);
3. **start time** (earlier first).

After a Deep (load 3) block, if the next slot is FREE, it becomes a **rest break** (15 min). The rest break belongs to the block: if the block moves, the break is released.

Why this order? Hard rules decide what is *allowed*. Urgency and importance decide the *order* of placement. Energy and rest decide *which slot*. This follows the idea of **hierarchical optimization** (§9.9): the goals are not treated as equally important.

6.5 Overload: "not enough time" vs. "no schedule found"

Two different messages must not be mixed.

Situation	Message	Why
Planned work due by a deadline is **more than** the free time before it	**"Not enough time"** — shortage shown in minutes.	This is a proof: it is a necessary condition.
The check passes, but the recipe cannot place every part	**"No schedule found"** — unplaced parts listed.	The recipe is a greedy method. If it fails, it does **not** prove that no plan exists.

When parts stay unplaced, the recipe uses the two-level protocol of the architecture (§7.6):

- **Level 1 — Sprint mode:** run again without rest breaks and with Shallow and low-priority (priority 1) tasks postponed. If every part is placed, show the schedule with a Sprint label.
- **Level 2 — Task triage:** place the most critical tasks first; list all unplaced tasks and the shortage; suggest choices (focus on the minimum required version, postpone a lower-impact task, ask the instructor for an extension 24–48 hours before the failure is expected).

6.6 Pseudocode

```
PLAN(tasks, profile, fixed_items):
  # Step 1 — Lock
  grid ← 672 slots; time the student confirmed as available → FREE, other time → UNKNOWN
  mark SLEEP slots (profile sleep window); mark fixed_items OCCUPIED
  set energy_level of every slot (default curve; replaced by the student's productive hours)
  mark each FREE run shorter than 30 min, with a non-free slot on both sides, as REST

  # Step 2 — Prepare
  for each task t that is not FIXED:
      factor    ← 1.35 if t.cognitive_load = 3 else 1.2
      t.planned ← ROUND_UP_TO_15(t.duration_min × factor)        # buffer applied once
      t.parts   ← SPLIT(t)                                        # 45–90 min parts
      t.score   ← importance(t) × urgency(t)
  queue ← tasks sorted by (score desc, deadline asc, list order); t after t.depends_on

  # Step 3 — Place
  for each deadline D:
      if SUM(planned of tasks due by D) > free time before D:  report "not enough time before D"
  for t in queue:
      for part in t.parts:
          allowed ← start positions s where
                    slots s .. s+len−1 are all FREE, end ≤ t.deadline, s ≥ end of t.depends_on
          if allowed is empty: add part to UNPLACED ; continue
          s ← position in allowed with the smallest (energy mismatch, neighbour score, start)
          mark slots as BLOCK
          if t.cognitive_load = 3 and next slot is FREE: mark next slot as BREAK

  # Step 5 — Report
  if UNPLACED is empty: return schedule
  else: SPRINT (no breaks; postpone Shallow and priority-1 tasks); if still not empty: TRIAGE
        return schedule + UNPLACED + shortage

REPAIR(schedule, new_event, now):                                 # Step 4
  mark the slots of new_event OCCUPIED
  affected ← blocks that overlap new_event          # all other blocks are frozen
  for b in affected (highest score first):
      move b to the FREE run nearest to its old start, not earlier than now,
      that still meets H1–H5
  if some block cannot move:
      unfreeze not-yet-started blocks of the lowest-score task, one task at a time, and try again
  if still not possible: add to UNPLACED and go to Step 5
  report stability = unchanged unaffected blocks ÷ all unaffected blocks
```

Cost: each part is compared with at most 672 slots, so Minh's 15 parts need about 15 × 672 ≈ 10,000 checks, instead of 10^{38} placements. This also supports the goal of under 2 seconds for 50 tasks (§9.4).

6.7 Worked example on Minh's week *(design check, not a benchmark)*

We re-computed this example by hand and with a small check script (`recipe_check.py`). Every number can be repeated by hand.

Step 2 result

Task	Estimate	Planned	Parts (min)	Score
MATH-1	120	165	90 + 75	0.088
LAB-1	60	75	75	0.086
PRJ-1 (code A)	120	165	90 + 75	0.001
PRJ-2 (code B)	120	165	90 + 75	0.001
PRJ-3 (test)	120	150 (load 2)	75 + 75	0.001

Task	Estimate	Planned	Parts (min)	Score
PRJ-4 (report)	180	255	90 + 90 + 75	0.001
PRJ-5 (slides)	60	75	75	0.001
LAUNDRY	45	60	60	0
CLEAN	30	45	45	0
Total	**855 min (14.25 h)**	**1,155 min (19.25 h)**	**15 parts**	

Math (due Wed 13:00) goes before the lab report (due Tue 23:59) by a small margin, because its priority is higher. Both are far ahead of the project parts, whose deadline is Sunday.

Step 3 result (capacity check passes: 1,050 min of work with a deadline vs. 5,100 min of free time before Sunday; the chores have no deadline)

Day	Time	Task	Length	Load
Mon	07:30–09:00	MATH-1	90	3
Mon	09:15–10:30	MATH-1	75	3
Mon	10:45–12:00	PRJ-1	75	3
Mon	17:00–18:15	LAB-1	75	2
Mon	19:15–20:30	PRJ-5	75	2
Mon	21:00–22:00	LAUNDRY	60	1
Mon	22:00–22:45	CLEAN	45	1
Wed	07:30–09:00	PRJ-1	90	3
Wed	09:15–10:45	PRJ-2	90	3
Fri	07:30–08:45	PRJ-2	75	3
Fri	09:00–10:30	PRJ-4	90	3
Fri	10:45–12:00	PRJ-4	75	3
Fri	17:00–18:15	PRJ-3	75	2
Fri	19:15–20:30	PRJ-3	75	2
Sat	07:30–09:00	PRJ-4	90	3

Result: **0 overlaps**, **all deadlines met**, nothing unplaced. Deep Work sits in energy-3 slots; Medium work in energy-2 slots; the chores are batched at the end of Monday (energy 1). A 15-minute rest break follows each Deep block, unless the next slot is already taken (for example by lunch). The test (PRJ-3) comes after both code tasks.

Step 4 result — a 2-hour event on Wednesday 08:00–10:00 (notified at 07:00)

Only the two blocks that overlap the event are affected (PRJ-1 07:30–09:00 and PRJ-2 09:15–10:45). PRJ-1 moves to Wednesday 10:00; PRJ-2 moves to Wednesday 17:00 (energy 2, so this soft rule is relaxed on purpose). The other **13 blocks stay unchanged: stability 13 / 13 = 100%**. The order test → after code is still respected.

Step 5 result — three overload cases (two Deep tasks due Tuesday 23:59)

Case	Planned work	Free time before the deadline	What happens
8 h + 10 h	1,470 min	1,365 min	The check **proves** a shortage of 105 min (1 h 45 min) → "not enough time".
8 h + 7 h (with rest breaks)	1,230 min	1,365 min	The check passes, but 4 parts (315 min) stay unplaced → **"no schedule found"**.

Case	Planned work	Free time before the deadline	What happens
8 h + 7 h in Sprint mode	1,230 min	1,365 min	2 parts (150 min) stay unplaced.

The second and third rows are a useful result: the check says "enough time", but the greedy recipe still fails. This is why the message must be "no schedule found", and why the claim "infeasible" needs a proof (§6.5).

6.8 Trade-offs of the chosen recipe

Choice	Benefit	Cost
Greedy placement instead of an exact search	Fast, easy to explain ("why is this task here?"), same result every time	Not always the best, and can fail on a nearly full week (see the second overload case)
Energy match as a **soft** rule	Does not block deadlines in a heavy week	Some Deep Work may land in medium-energy slots
One duration buffer for all users	Simple; works from day one	Does not fit every person (§8.5, §9.9)
Local repair first	The student keeps most of the plan; builds trust	Slightly more complex than rebuilding everything
Exact solver only as a test reference	Proves or refutes the recipe on small cases	Extra work for the team

6.9 Weak points found by the walk-through (to improve in the next loop)

1. **Nearly full weeks.** Fixed part sizes leave gaps that no part fits. *Idea to test:* allow flexible part sizes (between `min_chunk_min` and the maximum focus) when parts stay unplaced.
2. **Work crowds the early week.** "Earlier is better" puts three Deep blocks (240 min) on Monday morning, and leaves Tuesday and Thursday almost empty. *Idea to test:* a soft limit on Deep Work per day. This must be checked with the preference metric (§9.4).
3. **Equal-score tasks.** Parts of the same project have almost the same score, so their order depends on the list order. This is fixed by a clear tie-break rule (done), but the *logical* order of project steps is only covered by `depends_on`.

6.10 Re-checking the failing test cases

Test case	Result
Math placed late at night (S1)	Math is placed Monday 07:30–10:30 (energy 3).
Peak slots run out (S2)	Energy match is soft. A Deep part can use an energy-2 slot (the repaired PRJ-2) and still be placed.
Plan breaks after an event (S3, Perspective 4)	Only 2 of 15 blocks move. Stability 100% in the example.
Work > free time (Perspective 4)	The check proves the shortage (1,470 vs. 1,365 min) and reports it.

6.11 Computational check

Characteristic	How Pillar 4 meets it
Combinatorial and logical	Every step is a rule: IF a position breaks a hard rule THEN it is not allowed; IF a part cannot be placed THEN list it.
Deterministic and executable	Fixed factors, fixed formulas and fixed tie-breaks: the same task rows and the same grid always give the same schedule.
Scalable	About (number of parts × 672) checks. The same recipe serves any student; more tasks only means more rows.
Bridge between human and machine	"Plan my week" becomes 5 steps with exact numbers. "Fix my plan" becomes: freeze, move only what is hit, widen only if needed.

Hand-off: the four pillars give us a design. Next we place it inside a system (§7), test its limits (§8), and measure it (§9).

7. System architecture and how it works

The four pillars gave us the design of the core. This section shows how the core sits inside the whole assistant.

7.1 AI is one part, not the whole system

We first looked at four simple designs and found a limit in each one.

Initial approach	Limit we found	What we do instead
1. Pure LLM prompting — ask a language model to make the whole schedule	Language models are weak at time arithmetic. They can invent overlaps, break sleep time and give different answers to the same question.	**Split the work.** The language model only talks with the student and reads text (**M1**). A deterministic engine (**M2–M4**) does all the time work, so every hard constraint is checked by code, not guessed.
2. One-size-fits-all — assume 8:00 AM is everyone's peak hour	Ignores night owls and different lifestyles.	Use the student's own **productive hours** (Pattern 3). Presets for groups of users come later (§7.5).
3. Keyword-based filtering — mark words like "TikTok" as entertainment	Ignores real study or research intent.	**Importance is set by the student.** The assistant may suggest, but does not decide from keywords (§8.3, Example 1). The keyword table is used only for the work type (Pattern 2).
4. Short-term daily view — accept only micro-tasks	Fails on long-term goals and long assignment rubrics.	The student can enter a task of any size; M1 proposes a split into micro-tasks. Reading syllabus or rubric PDFs and long-term goals are **extensions** (Stage 5, §9.10).

Design rule (from the course project): *AI is one computational component, not the architecture.* Use AI where language and uncertainty justify it. Use deterministic logic where correctness, constraints and repeatability matter more.

Part	Uses AI?	Why
M1 — Natural Language Parser	**Yes** (language model)	The student's words are ambiguous ("sometime this week", "Wednesday's quiz"). Understanding them needs language skill. The result is shown to the student to confirm.
M2 — Constraint and Grid Manager	No	Calendar arithmetic must be exact.
M3 — Deterministic Engine (Math Solver)	No	Same input must give the same schedule; every decision must be explainable.
M4 — Dynamic Local Repair Module	No	Repair must follow fixed rules to keep the plan stable.
Suggestions (priority, work type)	Optional	They are only suggestions; the student decides.

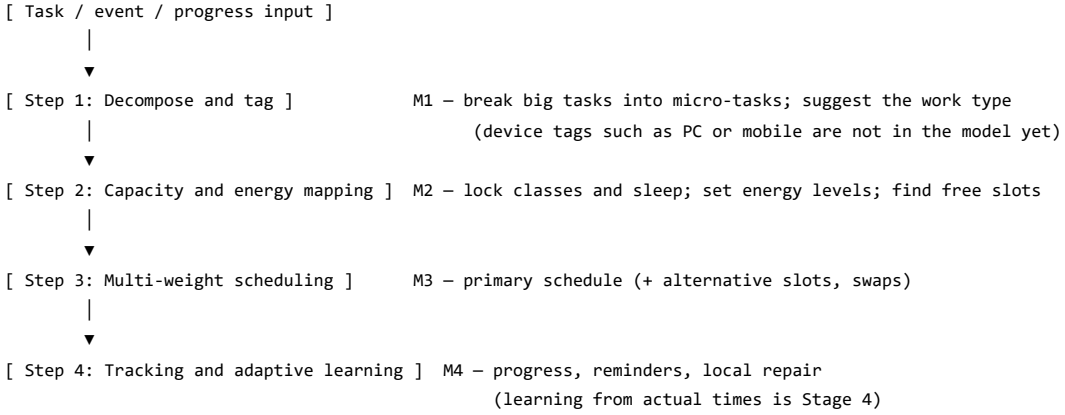
7.2 Inputs and outputs of the whole assistant

	Item	Description	Stage (§9.10)
Input	Profile and habits	Wake-up time, minimum sleep (hard lock), personal peak focus window. The student can see and change them on the screen at any time.	1
	Master Timetable (Recurring)	Reusable semester template for classes, fixed shifts, commute and sleep; inherited automatically each week (§7.8.1).	1
	Fixed commitments and tasks	One-time events or weekly tasks entered with forms or chat.	1
	Quick task	A chat sentence, for example "Schedule 1 hour of study this afternoon" (voice is optional).	1

	Item	Description	Stage (§9.10)
	Progress slider (%)	The student drags it to report progress. The remaining work is recalculated (the buffer is not applied a second time).	2
	Previous Week Retrospective	Feedback on last week's completion and fatigue to adjust pacing and motivation for this week (§7.8.4).	4 (extension)
	Document ingestion	Upload a syllabus, assignment rubric or slides; the system turns them into milestones.	5 (extension)
	Long-term goals	For example "Complete 10 practice exams in 2 weeks": spread over days.	5 (extension)
Output	Primary schedule	A visual time-block schedule.	1
	Overload warning	Conflicts and unplaced tasks.	1
	Batched shallow block	Micro-tasks under 20 minutes (one slot) are batched into one end-of-day block. Dead zones stay active rest (Pattern 5).	1–2
	Alternative slot options	2–3 backup slots the student can choose.	2
	Smart swap suggestions	When an urgent task arrives, propose swapping lower-priority tasks without breaking the plan.	2
	Motivation & pacing plan	Dynamic pacing adjustments based on previous week's stability (recovery vs. momentum pacing, §7.8.4).	4 (extension)
	7-day pending pool	Unconfirmed or postponed tasks are kept for 7 days so they are easy to reschedule.	2

The student keeps full control: the profile and constraints can be seen and edited at any time.

7.3 How a request flows through the system



The course project describes a pipeline *request → understand → route → retrieve/compute → reason → verify → respond*. Our system follows it:

Pipeline step	In our assistant
Request	A chat sentence, a form, a new event or a progress update.
Understand	M1 reads and checks the input; the student confirms.
Route	The routing table below.
Compute / Reason	M2 builds the grid; M3 orders by score and places (all deterministic).
Verify	Hard-rule check, capacity check, stability check.
Respond / Act	Schedule, warning or rescheduling plan.

Routing table (which part handles a request, and what happens when something is wrong):

Request	Route	If something goes wrong
New task (sentence or form)	M1 → student confirms → M3	A missing or invalid field → **ask the student** (or use a labelled default). An unknown work word → ask once.
New event	M4 (local repair)	No free slot → widen the repair; then Step 5.
Progress or duration change	M4 (update remaining work) → M3	—
Profile change	M2 (rebuild the grid) → M3	—
M1 fails or is too slow	**Fallback:** manual task form	—
Not enough time	Step 5 warning	Sprint mode, then triage (§7.6).

How the four steps of this flow, the four modules and the five recipe steps fit together:

Operating step	Module	Recipe steps (§6.3)
1 Decompose and tag	M1	Step 2 (prepare tasks)
2 Capacity and energy mapping	M2	Step 1 (lock)
3 Multi-weight scheduling	M3	Steps 3 and 5
4 Tracking and adaptive learning	M4	Step 4 (repair)

7.4 The four decision weights

The engine uses four criteria. They work in a fixed order of use, not as one big sum:

Weight	Principle	Effect	Used for
1. Urgency (deadline)	The closer the deadline, the faster the priority grows (exponential surge).	The task gets the earliest free slot.	**Order** of placement
2. Importance and context	High-stakes work (exams, major projects) goes first. The student defines importance; the assistant may suggest.	Important work is not pushed back by small tasks.	**Order** of placement
3. Energy fit (daily peak)	Demanding tasks go to peak focus hours; light work goes to low-energy times.	Better focus and a more realistic schedule.	Choice of **slot**
4. Burnout prevention (rest break and switch cost)	A 15-minute rest break follows Deep Work; jumps between very different types of work are avoided.	Less fatigue and fewer focus losses.	Choice of **slot**

Weights 1 and 2 become the **score** (§6.4). Weights 3 and 4 become the **energy mismatch**, the **rest break** and the **neighbour score**. The engine looks at all four criteria, and not at one sort key only. The hard rules come first and filter out every position that is not allowed.

7.5 Adapting to different groups of users

Different people need different settings. We keep the same model and only change the data and a few parameters.

Group	What is different	What the engine changes
University students (the first version)	Scattered credit hours; group projects; some are night owls.	Uses the student's own productive hours (including late-night windows); long rubrics are Stage 5.
K-12 students *(later)*	Rigid school hours; weaker time estimates.	Larger duration buffers; tight tracking of homework deadlines. The factor is a parameter to test (the ×1.35 of Pattern 1 is for hard tasks of any student).
Working professionals *(later)*	Fixed 8-hour work blocks.	Uses short off-hour windows; avoids fragmentation.

The first version serves **university students only** (§2.2). Office workers were rejected for the first version because of fixed shifts and little freedom; the other groups are **presets for Stage 4** and not part of the core.

7.6 Emergency response: Crunch Mode

Crunch Mode has two levels. It starts when parts stay unplaced (§6.5), or when an unexpected delay causes a deadline collision.

Level	Name	When	What it does
1	**Sprint mode**	The plan fits only if rest breaks are removed.	Remove non-essential rest breaks; postpone Shallow and low-priority tasks; use all open waking time for critical deadlines. (Sleep is never used.)
2	**Task triage**	Even Sprint mode cannot place everything, or the check proves a shortage.	**Minimum version:** focus on the core parts of the work (for example the main rubric items). **Strategic sacrifice:** suggest postponing lower-impact tasks to save the high-stakes one. **Early extension alert:** warn 24–48 hours before the failure is expected, so the student can ask for more time.

Fail-safe rule: when time is not enough, the engine never creates an impossible schedule. It gives clear options, and a human decides.

7.7 Reliability and safeguards *(planned)*

The course project asks for input checks, failure behaviour and privacy. Our plan:

- **Input validation.** M1's output is checked against the task model (type, multiple of 15 minutes, valid deadline) before use. Bad data → ask the student.
- **Failure behaviour.** If M1 fails or is too slow, use the manual task form. M2–M4 do not depend on a model.
- **Confirmation.** The student confirms the task rows and the schedule before they become active.
- **Privacy.** Keep only the data the model needs (§2.2, Discussion 3). Do not store real credentials in the repository, and do not send private data to outside AI services.
- **Document safety (Stage 5).** When PDFs are read, treat their text as *data*, not as instructions, and show the milestones for confirmation.

These points will be written in detail when we build the proof of concept.

7.8 Advanced System Extensions (Proposed Exploratory Directions)

> **Status:** Design proposals for potential future expansion (Stage 4, §9.10). These concepts sit outside the core 5-step recipe (§6) and represent candidate modular features to be investigated after the core deterministic engine is validated.

To keep the 5-step recipe (§6) robust, deterministic, and lightweight, our core scheduler focuses strictly on hard constraints and basic multi-weight placement. To address non-linear human psychology and administrative friction in real academic life, we outline four **proposed candidate extensions** that could sit outside the core engine in future iterations:

7.8.1 Reusable Recurring Commitment Master Template (Proposed Concept)

Problem: University students follow a fixed weekly timetable (lectures, labs, study shifts, commute, protected sleep ≥ 6 h, meals) that repeats across an entire 15-week semester. Forcing the student to re-enter these fixed commitments every Sunday causes severe user fatigue and abandonment.

Proposed Mechanism:

1. **Master Timetable Template:** At the beginning of the semester, the student would set up recurring commitments once (or import an `.ics` calendar file from the university portal `fitus.edu.vn` or Google Calendar).
2. **Weekly Inheritance:** At the start of each new week (Monday 00:00), M2 (Constraint and Grid Manager) could automatically clone the Master Template into the new week's 672-slot grid. The student would only need to enter new assignments and ad-hoc tasks for that week.
3. **Non-Destructive Exception Overrides:** If a holiday, class cancellation, or exam schedule alters a specific week, the student could click "Override for this week only." The modification would apply strictly to that specific week's grid, leaving the semester Master Template untouched for future weeks.

7.8.2 Human-in-the-Loop Conditional Task Decomposition (Proposed Concept)

Problem: An over-eager assistant that automatically splits tasks without domain context often invents artificial, unhelpful subtasks (e.g. chopping a cohesive 3-hour essay into three disconnected 1-hour chunks), confusing the student and breaking focus.

Proposed Interaction Principle:

1. **Avoid Blind Automatic Splitting:** As a fundamental design guideline, the assistant is intended *never* to fragment a task autonomously without explicit human collaboration and confirmation.
2. **Interactive Clarification Flow:** When a task exceeds the maximum continuous focus threshold (> 120 min) or cannot fit into any available continuous slot:
   - The assistant would prompt: *"This task is estimated at X hours. Would you like to break it into specific milestones, or schedule it as a single block?"*
   - **Condition to decompose:** Only if the student confirms and provides sufficient subtask details (e.g., "Part 1: Lit review (60m), Part 2: Code (60m)") or explicitly approves a proposed breakdown would M1 split it into micro-tasks.
   - **Fallback:** If the student declines or provides no details, the system would treat the task as an atomic block. If no single slot fits, it would transparently flag an unplaced block rather than secretly cutting it up.

7.8.3 Behavioral Dynamics & Psychological Energy Modeling (Exploratory Directions)

If pursued in Stage 4, human productivity modeling could account for four psychological dynamics:

1. **Warm-Up Latency & Activation Energy:**
   - Entering a Deep Work state requires $10 - 15$ minutes of cognitive ramp-up. Scheduling isolated, short 30-minute Deep Work blocks wastes half the session in activation friction.
   - A proposed extension could enforce a minimum Deep Work chunk size (e.g. ≥ 60 min) and use the neighbour score (w_4) to penalize rapid context switching between completely different subjects.
2. **Procrastination Priming & Momentum Building:**
   - When facing an intimidating, high-stakes Deep Work assignment, students frequently succumb to procrastination.
   - The assistant could offer an optional **Momentum Priming** arrangement: placing a 15-minute quick win (a Shallow or Medium task) immediately before the Deep Work block to build task-switching momentum (the Zeigarnik effect).
3. **Dynamic Cumulative Fatigue Tracking:**
   - Rather than treating energy as a static clock attribute, a candidate model could track cumulative Deep Work hours completed during the day.
   - After ≥ 3 hours of continuous Deep Work, the effective energy rating of subsequent afternoon slots could be dynamically discounted unless separated by a substantive break or meal block.
4. **Adaptive Peak Window Calibration:**
   - Over time, completion logs could help calibrate the student's actual productive hours, refining the difference between morning larks and late-night focusers without manual reconfiguration.

7.8.4 Cross-Week Retrospective & Motivation-Driven Adaptive Scheduling (Proposed Concept)

Problem: A static calendar treats each week in isolation. If a student finishes an exhausting exam week, scheduling maximum workload on Monday morning leads to immediate burnout and schedule abandonment.

Proposed Mechanism:

1. **Sunday Evening Retrospective (Proposed Previous Week Health Score):** The system could compute an exploratory retrospective score based on:
   - *Task Completion Rate:* Proportion of planned tasks successfully completed.
   - *Disruption & Crunch Frequency:* Number of emergency local repairs or Sprint Mode activations.
   - *Self-reported Fatigue (EMA):* End-of-week exhaustion level (1 to 5).
2. **Motivation-Driven Pacing Options for the New Week:**
   - **Scenario A — Post-Crunch Recovery:** If the previous week was overloaded (Crunch Mode triggered or fatigue $\geq 4/5$), the new week could automatically expand duration buffers by $+10\%$, insert mandatory buffer zones, and front-load achievable Quick Wins on Monday–Tuesday to rebuild psychological confidence.
   - **Scenario B — Flow & Momentum Maintenance:** If the previous week showed high completion ($> 85\%$) with low friction, the assistant could maintain optimal pacing and schedule major project milestones during peak windows.
   - **Scenario C — Chronic Disruption Zones:** If past weeks reveal recurring emergency interruptions at specific times (e.g. recurring Thursday evening group meetings), the system could proactively mark those periods as reserved buffer zones.

8. Edge cases and design trade-offs

8.1 Why we look at edge cases

A schedule that looks reasonable can still fail in unusual cases. We looked for such cases early, and used them to change the design. This is the same loop as in the pillars: *first idea → failure case → revised idea*.

8.2 Ten edge cases and how we handle them

#	Edge case	Possible problem	How we handle it	Where / test
1	Incomplete schedule information	The student forgot a class; the assistant uses that time.	Initial setup; recurring events; a warning that the schedule depends on the information given.	M2; T07
2	Ambiguous task importance	The same activity can be important or not, depending on purpose.	The student sets importance (and may add context). No classification from a title alone.	§8.3 Ex. 1; T08
3	Uncertain task duration	A "2-hour" task may need 5 hours.	Duration buffer; duration updates; optional ranges; replan the remaining work.	Pattern 1, M4; T05
4	Excessive workload	Work is more than free time before the deadlines.	Keep hard constraints; report unscheduled tasks and the shortage.	Step 5; T06
5	Conflicting deadlines	Several high-priority tasks are due together.	Check feasibility first; then prioritize in a way the student can see.	Step 3, score; T02
6	Unsplittable task	A 2-hour block is needed but only short gaps exist.	Place only in a long enough slot, or say that no suitable slot exists.	H2; T03
7	Unexpected event	A new meeting overlaps a study session.	Move or split the affected work where allowed; change as little as possible.	Step 4; T04
8	Individual differences	Two students need different time for the same task.	Use each student's own estimates and profile; do not assume equal speed.	Profile; T09
9	Task crosses the planning boundary	A task due next week needs work this week.	Consider deadlines after the displayed week (extra week in the grid).	§5.5; T10
10	Missing or invalid details	No deadline, unrealistic duration or contradictory data.	Ask for what is essential, use labelled defaults, or flag for review.	M1 validation; T07

8.3 Four worked examples

Example 1 — Subjective importance. Two students enter the same activity: *"Watch TikTok for one hour."* Student A wants entertainment. Student B researches video-editing for a media project.

First approach	Limit	Revised approach	Design insight
Sort tasks into important / unimportant from their titles.	The title does not show the student's purpose.	The student says the purpose or importance. The assistant may suggest; the student decides.	Importance is not an objective property that can always be read from text. Urgency can be estimated from deadlines and remaining work; importance depends on personal goals.

Example 2 — Wrong duration. A student enters a programming assignment: 2 hours, due Friday 23:59. The system places 2 hours on Thursday. After 2 hours, the student needs 3 more.

First approach	Limit	Revised approach	Design insight
Treat the estimate as fixed.	Students underestimate unfamiliar or complex tasks.	Track remaining work; let the student change the estimate; update later sessions and show new conflicts. For large tasks, offer a split into subtasks.	An estimate is information that can be updated. It is not a promise.

Example 3 — Not enough time. The student has 6 hours free before Friday. Assignment A needs 4 hours and B needs 3 hours. Both are due Friday. The total (7 h) is 1 hour more than the free time.

First approach	Limit	Revised approach	Design insight
Try to fit all tasks.	Some problems have no solution under the given constraints.	Check feasibility before showing a schedule as complete. If it is not feasible, show the shortage and offer changes (priorities, estimates or more available time).	An honest warning is more useful than a schedule that looks complete but is not realistic.

Example 4 — Recovering from a disruption. A two-hour project session is planned 14:00–16:00. A meeting is added 14:30–16:30.

First approach	Limit	Revised approach	Design insight
Rebuild the whole week after every change.	Many unrelated sessions move; the student gets confused and trusts the schedule less.	Keep completed, protected and unaffected blocks. Reschedule only the work that must change. Widen the repair only if needed.	A good rescheduling system optimizes feasibility **and** stability.

8.4 Design alternatives and the decision for each

We compared the options by **reliability, user effort, complexity and future growth**, instead of choosing at once.

Design question	Alternative A	Alternative B	Decision
How should tasks be classified?	Infer all priorities automatically.	Ask the student for priorities and context.	Student-defined priorities, with optional AI suggestions.
How much should the student enter?	A full daily activity log.	A small set of essential inputs.	Minimal required input, with optional details and updates over time.
How is duration estimated?	Predict it from the task description.	Ask for estimates and improve them with feedback.	Start with the student's estimates; consider personal estimation later.
How are disruptions handled?	Rebuild the whole schedule.	Change only the affected blocks.	Local repair first; wider replanning only when needed.
Should we support long-term goals?	Yes, goals of months or years, now.	First focus on concrete weekly tasks.	Build reliable weekly scheduling first; then add goal decomposition.
How is personal capacity handled?	Assume the same capacity for everyone.	Use personal preferences and updated estimates.	Allow customization; explore feedback-based adaptation later.

8.5 Four deeper trade-offs

Automation vs. user control. More automation saves effort but risks wrong assumptions (for example, labelling every activity automatically). We follow a **human-in-the-loop** approach: the assistant recommends; the student controls important assumptions, constraints and final changes. This needs a little more interaction at the start, but it is clearer and it limits the cost of a wrong automatic decision.

Input completeness vs. convenience. Detailed input makes better schedules, but asking for every small activity is too much work. Our setup is light: (1) recurring commitments and protected time; (2) tasks with the essential scheduling data; (3) extra preferences only when needed; (4) quick updates when plans change. An **empty calendar does not mean free time**: the model must tell *confirmed available* time apart from time that is *not yet confirmed* (§5.5, state UNKNOWN). The aim is to **reduce input effort without hiding uncertainty**.

Weekly tasks vs. long-term goals. "Finish the English vocabulary assignment by Friday" is a task. "Reach IELTS 7.0 in one year" is a goal that needs the student's current level, milestones, effort over months and progress checks. That is a learning-planning problem, not a scheduling problem. The first version schedules clear tasks in a **weekly** horizon. A later version may turn a goal into measurable milestones and then schedule their tasks.

Personalization vs. prediction reliability. A fixed model may suit some students badly, but predicting personal capacity needs history that a new student does not have. We grow step by step: *start* with the student's own estimates and preferences; *then* let the student report actual times; *later* explore personal estimation from the history. Predictions stay adjustable and never override a clear decision by the student.

8.6 Final design direction

We propose a **constrained, user-centred** scheduling assistant with three principles:

Principle	Meaning	Where it appears
1. Feasibility before optimization	Respect hard constraints; tell a feasible plan from an unresolved conflict.	Hard rules H1–H5; capacity check; "not enough time" vs. "no schedule found" (§6.5)
2. Transparency before excessive automation	The student understands why a task is where it is, and can correct inputs and priorities.	Simple score and rules; student confirms M1; human-in-the-loop
3. Stability before unnecessary replanning	A surprise triggers a targeted repair, not a new plan.	Step 4; stability metric (§9.5)

The result is a **scheduling assistant, not an authority over the student's personal life**.

8.7 Decision flows (The Continuous Design Refinement Loop)

A hallmark of Computational Thinking is the **iterative improvement loop**: *Initial intuitive idea* $\rightarrow$ *Failing edge case* $\rightarrow$ *Revised computational design*. Below, we trace how our core architectural decisions evolved when evaluated against realistic student failure scenarios:

1. Summary Matrix of Design Evolution

Decision	Initial Idea (Intuition)	Failing Edge Case (Why it broke)	Revised Computational Design	Core Design Insight
Decision A: Task Importance	Automatically infer task importance from title keywords.	Identical titles carry opposite meanings (e.g., *"Watch TikTok"* can be leisure or project research).	**Student-defined importance** + optional AI recommendation + manual override (§8.3 Ex. 1).	Importance is subjective and intent-dependent; urgency is mathematical.
Decision B: Disruption Recovery	Full regeneration: rebuild the entire weekly schedule upon any delay.	Unrelated study blocks move across the week, disorienting the student and destroying trust.	**Targeted local repair (M4):** freeze unaffected blocks; relocate only impacted blocks (§8.3 Ex. 4).	Rescheduling must optimize stability ($\geq 90\%$) alongside feasibility.
Decision C: Division of Labor (AI vs. Code)	"Pure LLM prompting": prompt a single language model to generate the whole timetable.	Hallucination and arithmetic errors: overlapping slots, broken sleep windows, inconsistent rules.	**Decoupled architecture:** LLM reads messy language (M1); deterministic engine does grid math (M2–M4).	AI is one computational component, not the system architecture.

2. Detailed Decision Refinement Flows

Decision A — Task Importance Calibration

- **Initial Idea:** Infer importance automatically from activity keywords.
- **Failure Case:** Activity titles cannot convey student intent. A student entering *"Watch TikTok"* could be procrastinating or conducting competitive research for a digital marketing assignment.
- **Revised Design:** Importance is explicitly designated by the student (scale 1 to 5). The assistant may suggest a baseline based on academic keywords, but the human retains complete authority.
- **Key Lesson:** Importance is subjective and personal; only urgency can be computed mathematically from deadlines and remaining work.

Decision B — Disruption Recovery Strategy

- **Initial Idea:** Trigger a complete schedule rebuild whenever an unexpected delay or appointment arrives.
- **Failure Case:** A minor 1-hour delay on Tuesday afternoon causes Friday morning study sessions to jump around, violating user mental models and causing schedule abandonment.
- **Revised Design: Dynamic local repair (Step 4, M4)**. All past, locked, and unaffected blocks remain frozen. Only the overlapping blocks are moved to the nearest feasible free slots. The repair scope is widened only if a feasible deadline cannot be preserved.
- **Key Lesson:** A good rescheduling assistant prioritizes **schedule stability ($\geq 90\%$)** to preserve human trust.

Decision C — Role of Artificial Intelligence vs. Deterministic Logic

- **Initial Idea:** Prompt a Large Language Model with the student's tasks and calendar to generate the schedule directly.
- **Failure Case:** Language models are notoriously poor at discrete constraint satisfaction and interval arithmetic. They frequently hallucinate non-existent time slots, overlap with classes, and violate protected sleep.
- **Revised Design: Strict separation of concerns**. The language model is confined strictly to natural language parsing (M1). All calendar grid calculations, constraint validations, and slot placements are executed by a deterministic, zero-hallucination math solver (M2–M4).
- **Key Lesson:** AI is one component for natural language understanding, not the scheduling architecture itself.

9. Evaluation plan

> **Status:** this is a **plan**. All targets are **goals**, not achieved results.

9.1 Objectives and evaluation questions

The main objective: find out whether the assistant gives a **more reliable and more convenient** scheduling experience than normal manual planning. A good assistant must not only make a schedule. It must make **valid** schedules, react to disruptions, find impossible workloads and reduce the student's effort.

ID	Question	Link
EQ1 — Schedule correctness	Can the system make schedules without breaking hard constraints?	§6, §8.6
EQ2 — Scheduling effectiveness	Can it place tasks before their deadlines when a feasible schedule exists?	§6
EQ3 — Rescheduling stability	Can it recover from disruptions without changing unrelated activities?	Step 4
EQ4 — User experience	Does it reduce manual scheduling effort and give clear, trustworthy recommendations?	§8.5
EQ5 — Robustness	How does it behave when input is incomplete, unclear, wrong or impossible?	§8.2

9.2 Scenario-based testing

We build a controlled set of scenarios, from normal to hard, and include **cases with no feasible schedule** (finding them is a central requirement).

Test	Scenario	Expected outcome	From
T01	Five tasks with enough free time.	All tasks placed before their deadlines; no hard constraint broken.	UC1, UC2
T02	Several tasks with overlapping deadlines.	A feasible schedule if one exists; otherwise a clear warning.	UC3
T03	A large task with only short free periods.	Split into valid sessions if allowed; otherwise leave unplaced and explain.	UC4, UC6
T04	A new event overlaps a scheduled task.	Repair the affected part; keep other blocks where possible.	UC8
T05	A task needs more time than estimated.	Update the remaining work and revise the schedule.	UC9
T06	Total work is more than the available time.	Report unscheduled work and deadline conflicts; protected time is not touched.	UC10
T07	The student gives incomplete schedule information.	Ask for or flag the missing essential information; do not assume the student is free.	Edge cases 1, 10
T08	A task has an unclear priority.	Keep the student's chosen priority, or ask.	Edge case 2
T09	Two students give the same tasks but different preferences.	Same hard constraints satisfied; different preferences reflected.	UC7
T10	A deadline is after the current week.	Place the right amount of this week's work, considering the later deadline.	Edge case 9

Minh's week (§6.7) can be the first test data: the normal week, the Wednesday event and the two overload cases cover T01, T04 and T06.

How we check results. For small scenarios, feasibility can be checked by hand or with an **independent constraint solver** that gives a reference answer. For larger scenarios, the schedule is checked automatically for overlaps, deadline misses and other hard-constraint failures.

9.3 Comparative testing and the improvement plan

We compare three ways to schedule, using **the same tasks, constraints and scenarios**, so that differences come from the method and not from the input.

Approach	Description	Purpose
Baseline A — Manual scheduling	The student builds and edits a schedule with a normal calendar or to-do list.	Measure real effort and usability.
Baseline B — Simple rule-based scheduling	Tasks are placed by a basic rule, such as Earliest Deadline First, inside the available time.	Show whether our extra logic adds value.
Proposed — Constraint-based adaptive scheduling	Uses hard constraints, preferences, task structure and schedule stability.	Evaluate the complete idea.

An important comparison: **full regeneration** vs. **local repair** after an unexpected event.

Improvement cycle (plan). The three strategies of Pillar 4 give a natural order of versions. We run **the same tests (T01–T10) and the same metrics** for each version, and record before/after results.

Version	What it adds	Metric we expect to change (hypothesis)
V0	Baseline B: order by deadline; earliest free slot (strategy S1).	Starting point. Good deadlines on easy tests; low preference satisfaction.
V1	Priority score + Deep/Shallow type + energy match (S2, Patterns 2–3).	Preference satisfaction goes up.
V2	Duration buffer + rest break + batching (S3, Patterns 1, 4, 5).	Fewer reschedules in the sensitivity test.
V3	Local repair + overload messages (S3, Step 4–5).	Stability reaches the goal; conflict reporting reaches 100%.

These are **hypotheses to test**, not results. After every version, we run all earlier tests again (regression), so that a fix does not break something that worked.

9.4 Quantitative metrics (goals)

Metric	Measurement	Goal
Hard constraint violations	Overlaps with fixed commitments, protected sleep or other forbidden time.	**Zero** in every schedule.
Deadline satisfaction rate	% of tasks whose work finishes by the deadline.	100% on instances confirmed feasible.
Infeasibility detection accuracy	% of cases correctly marked feasible or infeasible against a verified reference.	100% on the core verified test set.
Conflict reporting coverage	% of unscheduled tasks and unresolved deadline conflicts correctly reported.	100% on the verified test set.
Schedule stability	% of previously scheduled, unaffected blocks that stay unchanged after rescheduling.	At least 90% where feasible.
Preference satisfaction	Share of eligible study time in preferred focus periods (can be weighted by importance).	Better than the simple rule-based baseline.
Scheduling execution time	Time to build or repair a schedule for a given number of tasks and slots.	Under 2 seconds for a 50-task week on stated hardware.
Manual planning effort	Time or number of actions to build and update a schedule.	Lower than manual scheduling.

> All targets are proposed goals, not achieved results.

Two notes on how we read these goals:

- The recipe is **greedy** (§6.8). If it misses a feasible instance, we do not hide it. It is logged as an error and gives an idea for the next version (for example flexible part sizes, §6.9).
- The product says "infeasible" only with a **proof** (§6.5). Cases of "no schedule found" are counted **separately** from cases proved infeasible.

9.5 Schedule stability

Stability matters most when something unexpected happens.

```
Schedule Stability = (Unchanged Unaffected Blocks ÷ Total Previously Scheduled Unaffected Blocks) × 100%
```

Unaffected blocks are those that do not directly overlap the new disruption. Their old times are compared with the new schedule.

Example. A weekly schedule has 20 study blocks. A new event overlaps 2 of them, so 18 are unaffected. If 17 of these 18 stay unchanged, stability = 17 ÷ 18 × 100% ≈ **94.4%**, which meets the 90% goal.

Priority rule. The stability goal does **not** apply when keeping blocks would make a feasible deadline impossible. **Hard constraints and deadline feasibility come before stability.**

9.6 Scale and sensitivity tests

- **Scale.** Run with 10, 25, 50 and 100 tasks; record time and quality.
- **Estimate errors.** Increase task durations by **25%** and **50%** in controlled scenarios, and see how often the system must reschedule and whether it reports the new conflicts. This is linked to the buffers: +25% is more than the ×1.2 buffer but inside the ×1.35 buffer; +50% is above both. So the test shows what the buffers cannot protect.

9.7 In-Silico Behavioral Simulation: The SimulatedStudent Model (Proposed Methodology)

> **Status:** Proposed evaluation methodology. This is a conceptual in-silico simulation design intended to stress-test human behavioral assumptions without ethical risk or human participant burden. Implementation is planned if computational and modeling resources permit during Stage 3 (§9.10).

While unit scenarios (T01–T10) verify mathematical correctness, evaluating real-world performance requires testing how the assistant responds to messy human behavior. Gathering hundreds of human participants for edge-case testing is impractical and raises ethical concerns (e.g. intentionally overloading students during finals).

To explore this evaluation direction, we propose a candidate **in-silico agent-based behavioral simulation** framework. Instead of guessing arbitrary human actions, we outline a candidate `SimulatedStudent` class grounded in **established empirical literature** from biomedical chronobiology, cognitive psychology, and econometrics.

1. Scientific Foundations of the Proposed Simulation Model

Domain	Proven Scientific Law	Mathematical / Statistical Implementation in `SimulatedStudent`
Biomedical & Chronobiology	**Circadian Rhythm & Chronotype Distribution** (Roenneberg et al., Munich ChronoType Questionnaire - MCTQ)	Individual cognitive peak hour follows a normal distribution: $H_{\text{peak}} \sim \mathcal{N}(\mu = 10.5\ \text{h}, \sigma = 2.0\ \text{h})$, truncated to waking hours $[07:00, 23:00]$. Circadian alignment for slot t is: $\text{CircFit}(t) = \max\left(0, 1 - \frac{\|$
Biomedical & Physiology	**Ultradian Cycles & Cognitive Fatigue Accumulation** (Kleitman's Basic Rest-Activity Cycle; Baumeister's Ego Depletion)	Sustained Deep Work depletes mental energy exponentially: $E(t) = E_0 \cdot \exp(-\lambda \cdot t)$, with depletion rate $\lambda \sim \mathcal{N}(0.15, 0.03)\ \text{h}^{-1}$. A 15-minute rest break (Pattern 4) restores energy by $\Delta E = 0.40 \cdot (E_{\max} - E(t))$.
Cognitive Psychology & Behavioral Economics	**Planning Fallacy & Overconfidence Bias** (Kahneman & Tversky, 1979; Buehler et al., 1994)	Estimation error is asymmetric: humans rarely overestimate duration but systematically underestimate by 20–40%. Actual duration follows a **Log-Normal distribution**: $T_{\text{actual}} = T_{\text{estimate}} \cdot \exp(X)$, where $X \sim \mathcal{N}(\mu = 0.25, \sigma = 0.20)$. The median inflation factor is $\exp(0.25) \approx 1.28$, directly validating our Pattern 1 buffers ($\times 1.20$ to $\times 1.35$).
Behavioral Economics	**Procrastination & Temporal Motivation Theory** (Steel, 2007;	Baseline procrastination susceptibility trait $P_{\text{trait}} \sim \text{Beta}(\alpha = 2, \beta = 5)$. Deadline urgency decays hyperbolically with slack: $\text{Urgency}(t) =$

Domain	Proven Scientific Law	Mathematical / Statistical Implementation in `SimulatedStudent`
	Ainslie's Hyperbolic Discounting)	$\frac{1}{1+\Gamma \cdot \text{SlackHours}(t)}$. When slack is large, the student is prone to delay study sessions.
Econometrics	**Discrete Choice & Random Utility Model** (McFadden, 1974 - Nobel Laureate)	At each scheduled block, the agent chooses between two alternatives: **Adhere to the block** vs. **Yield to distraction / Procrastinate**. The net utility difference determines the choice via a logistic choice model with random noise.
Stochastic Processes	**Disruption Arrival Model** (Poisson Point Process)	Unplanned external emergencies (illness, urgent family errand) arrive as a Poisson process with mean rate $\lambda_{\text{disrupt}} \sim \mathcal{N}(0.40, 0.15)$ events/day, with duration $D_{\text{disrupt}} \in \{60, 90, 120\}$ min.

2. Illustrative Implementation: The `SimulatedStudent` Class

The following Python class illustrates how these behavioral principles could be formalized into an executable simulation agent for future experimentation:

```python
import random
import math

class SimulatedStudent:
    """
    Simulates a university student's behavioral, physiological,
    and psychological responses to an automated weekly schedule.
    Grounded in chronobiology, Kahneman-Tversky planning fallacy,
    Steel's temporal motivation theory, and McFadden's random utility model.
    """
    def __init__(self, student_id: str, seed: int = None):
        if seed is not None:
            random.seed(seed)
        self.student_id = student_id

        # 1. Chronobiology: Peak cognitive focus hour (Normal distribution, clipped to 7:00-23:00)
        self.peak_hour = max(7.0, min(23.0, random.gauss(10.5, 2.0)))

        # 2. Planning Fallacy: Log-normal duration bias parameter (median exp(0.25) ≈ 1.28)
        self.duration_bias = max(0.95, min(2.10, math.exp(random.gauss(0.25, 0.20))))

        # 3. Behavioral Trait: Procrastination susceptibility (Beta distribution, mean ≈ 0.28)
        self.procrastination_trait = random.betavariate(2, 5)

        # 4. Biomedical Fatigue: Depletion rate per hour of continuous Deep Work
        self.fatigue_rate = max(0.08, min(0.25, random.gauss(0.15, 0.03)))
        self.current_fatigue = 0.0  # Scales from 0.0 (fresh) to 1.0 (exhausted)

        # 5. Stochastic Life Disruptions: Poisson event rate (events per day)
        self.disruption_rate = max(0.10, random.gauss(0.40, 0.15))

    def calculate_actual_duration(self, estimated_minutes: int) -> int:
        """
        Calculates actual execution duration using Kahneman-Tversky log-normal bias.
        Rounds up to the 15-minute grid resolution.
        """
        noise = random.gauss(1.0, 0.08)
        actual = estimated_minutes * self.duration_bias * noise
        return max(15, int(math.ceil(actual / 15.0) * 15))

    def evaluate_block_adherence(self, slot_hour: float, task_importance: float,
                                 slack_hours: float) -> bool:
        """
        McFadden Random Utility Model (RUM): Computes whether the student
        actually starts the scheduled task or yields to procrastination.
        """
        # Circadian alignment (distance to peak focus hour)
        circadian_fit = max(0.0, 1.0 - (abs(slot_hour - self.peak_hour) / 8.0))

        # Temporal Motivation Theory: Urgency increases as slack shrinks
        urgency = 1.0 / (1.0 + max(0.1, slack_hours / 24.0))

        # Utility of studying
        u_study = (0.35 * circadian_fit +
                   0.30 * task_importance +
                   0.25 * urgency -
                   0.20 * self.current_fatigue)

        # Utility of distraction with stochastic Gaussian noise
        noise = random.gauss(0.0, 0.05)
        u_distract = self.procrastination_trait + noise

        # Logistic choice probability
        prob_adhere = 1.0 / (1.0 + math.exp(-4.0 * (u_study - u_distract)))
        return random.random() < prob_adhere
```

```
    def apply_study_fatigue(self, duration_minutes: int, is_deep_work: bool,
                            has_rest_break: bool):
        """
        Updates cognitive fatigue state based on session intensity and breaks.
        """
        if is_deep_work:
            hours = duration_minutes / 60.0
            self.current_fatigue = min(1.0, self.current_fatigue + self.fatigue_rate * hours)
            if has_rest_break:
                # 15-min rest break restores 40% of accumulated fatigue
                self.current_fatigue = max(0.0, self.current_fatigue * 0.60)
        else:
            # Shallow work causes minimal fatigue (1/3 of Deep Work)
            self.current_fatigue = min(1.0, self.current_fatigue + (self.fatigue_rate * 0.3) * (duration_minutes / 60.0))

    def step_nightly_recovery(self, sleep_hours: float):
        """
        Restores cognitive energy after sleep. Sleep < 6h leaves residual fatigue.
        """
        if sleep_hours >= 7.0:
            self.current_fatigue = 0.0
        else:
            # Sleep penalty: each hour below 7h leaves residual fatigue
            deficit = max(0.0, 7.0 - sleep_hours)
            self.current_fatigue = min(0.60, deficit * 0.20)
```

3. Proposed Monte Carlo Simulation Protocol (Candidate Design)

If computational resources allow, this `SimulatedStudent` model could form the basis of a **Monte Carlo experiment** simulating $N = 1,000$ virtual students across $10,000$ simulated weeks:

1. **Proposed Cohort Stratification:**
   - **Group 1 — Light Load (30%):** 10–15 h of weekly academic tasks; balanced sleep habits.
   - **Group 2 — Typical Load (50%):** 20–25 h of weekly tasks; mix of early birds and night owls.
   - **Group 3 — Heavy / Crunch Load (20%):** 30–40 h of weekly tasks; high risk of overload.
2. **Simulation Lifecycle:**
   - On Monday 00:00, the assistant generates the week's schedule for each agent.
   - The simulation advances time slot by slot (15-minute resolution).
   - In each slot, the agent evaluates task adherence via the McFadden choice function, experiences task duration inflation via the Kahneman-Tversky log-normal bias, accumulates fatigue, and triggers Poisson disruption events.
   - When disruptions occur, M4 Local Repair is invoked, and the simulation logs whether the repaired schedule prevents deadline violations.
3. **Key Hypotheses to Test in Simulation (Hypotheses, not achieved results):**
   - **Buffer Sufficiency Hypothesis:** The $\times 1.2$ and $\times 1.35$ duration buffers absorb at least 85% of empirical duration overruns without requiring a schedule rebuild.
   - **Stability Advantage Hypothesis:** M4 Local Repair achieves $\geq 90\%$ block stability compared to $< 45\%$ for full global recalculation under Poisson emergency arrivals.
   - **Rest Break Necessity Hypothesis:** Agents scheduled without 15-minute rest breaks experience cognitive fatigue collapse ($E > 0.85$) by Thursday afternoon, resulting in a $> 40\%$ drop in Friday task adherence.

9.8 In-Vivo Longitudinal Field Study: Proposed Peer Cohort Design

> **Status:** Proposed user study protocol for Stage 3 evaluation (§9.10). All participant counts, protocols, and questionnaire targets represent proposed evaluation plans and goals, not completed empirical studies.

To complement synthetic simulation with authentic human judgment, we propose a tentative **1-week longitudinal field study design** with a small cohort of real students.

1. Participant Selection: The Proposed Peer Cohort Approach

Rather than attempting an uncontrolled public survey, we propose recruiting $N \approx 10$ to 14 **university students who are personal acquaintances and classmates of the group members** across HCMUS programs (Advanced Program in Computer Science, Computer Science, Data Science, Information Systems).

Methodological rationale for this proposed sampling approach:

- **High completion potential:** In mobile and productivity app studies, stranger dropout rates often exceed 60%. Close peers are significantly more committed to logging complete data across the entire 7 days.
- **High-fidelity qualitative feedback:** Friends can communicate candidly about friction points, confusing UI outputs, and unrealistic suggestions without social desirability bias.
- **Controlled demographic context:** All participants share the same institutional environment (FIT-HCMUS), calendar structure, assignment types, and commute realities, matching our problem boundary (§2.2).

2. Proposed 7-Day Study Protocol (Monday 00:00 to Sunday 23:59)

If approved and conducted, the study would follow a structured 3-phase timeline:

Phase	Timeline	Proposed Activities & Instruments
Phase 0: Pre-Study Onboarding	Sunday (Day 0)	- Input recurring commitments (classes, lab shifts, meals, commute). - Complete Munich ChronoType Questionnaire (MCTQ) self-assessment to calibrate individual baseline energy curves (M2). - Set non-negotiable sleep windows (hard constraint ≥ 6 h).
Phase 1: Daily Operational Cycle	Days 1–7 (Mon–Sun)	- **Morning (07:00–09:00):** Student inputs the day's tasks via natural language text or voice; reviews and confirms M1's extracted task rows. - **Daytime Execution:** Student works according to the schedule. - **Real-Time Disruption Logging:** When unexpected events occur (e.g. urgent meeting, lab overrun), the student logs the disruption immediately; M4 generates a local repair. - **Nightly Ecological Momentary Assessment (EMA):** A 3-minute check-in form via mobile chat.
Phase 2: Post-Study Evaluation & Debriefing	Sunday evening (Day 7)	- Complete standardized **System Usability Scale (SUS)** survey. - Complete **NASA-TLX** cognitive workload comparison (comparing the assistant vs. manual planning). - 30-minute semi-structured qualitative interview.

3. Proposed Evaluation Instruments and Data Collection

1. **Daily Ecological Momentary Assessment (EMA) Questions (1–5 Likert Scale):**
   - *Adherence:* "What percentage of scheduled study blocks did you complete today?" ($< 50\%, 50-75\%, 75-90\%, 90-100\%$).
   - *Cognitive State:* "How mentally exhausted do you feel right now?" ($1 = \text{Fresh}, 5 = \text{Completely Burned Out}$).
   - *Perceived Realism:* "Did the scheduled study times match your actual energy levels today?" ($1 = \text{Strongly Disagree}, 5 = \text{Strongly Agree}$).
   - *Disruption Impact:* "If your schedule changed today, did the repair feel natural or jarring?"
2. **Post-Study Standardized Questionnaires:**
   - **System Usability Scale (SUS):** 10-item standard questionnaire. Goal: score $\geq 75/100$ (well above the industry average of 68).
   - **NASA-TLX (Cognitive Workload):** Measures 6 sub-scales: Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration. Goal: lower Mental and Temporal Demand compared to manual planning.
3. **Semi-Structured Debriefing Interview Questions:**
   - *"Did you trust the assistant when it warned you that you did not have enough time?"*
   - *"When an emergency happened, did the local repair save your week, or did you abandon the schedule?"*
   - *"Were the 15-minute rest breaks helpful, or did they interrupt your flow?"*
   - *"What was the most frustrating assumption the assistant made about your time?"*

9.9 Error taxonomy and regression tests

To learn from failures we sort them into types. This is our **start list**; we will change it after the first test runs.

Code	Error type	Example
E1	Hard-constraint violation	Overlap, sleep used, deadline missed while a feasible plan exists.
E2	Wrong feasibility message	Says "infeasible" without proof; or says "feasible" but leaves a task unplaced.
E3	Unstable repair	Moves blocks that the event did not touch.
E4	Poor preference fit	Deep Work in low-energy slots when a peak slot was free.
E5	Parsing error	M1 puts a wrong value in a field.
E6	Missing question	The assistant does not ask for essential missing data.

Regression tests. Every fixed error becomes a new test. After each change, we run all tests again.

9.10 Roadmap and addressing current challenges

1. Current Challenges and Proposed Directions

#	Challenge	Why it matters	Direction
1	**Dependence on user information**	A schedule is not reliable if important commitments are missing; but asking for everything is not practical.	Recurring commitments, small updates, clear warnings about assumptions. Calendar link later.
2	**Subjective importance**	Importance depends on the student's intent and goals.	Separate **urgency** (calculated from deadline and remaining work) from **importance** (mostly set by the student, editable).
3	**Task complexity and duration**	Difficulty varies by subject, content and person. A short description does not show the real time.	Start with the student's estimates; optional subtasks; collect completion feedback. Personal prediction when there is enough data.
4	**Changes in a person's capacity**	Productivity differs between people and over time. A constant speed may overload some students.	Let students set session lengths and workload; later suggest changes from history, with the student's approval.
5	**Competing goals**	A very stable schedule may not meet deadlines or preferences best; maximizing preferred time may cut work into small pieces.	A **hierarchical optimization** strategy (below).
6	**Cold start and long-term goals**	A new student has no history; goals like an IELTS score are not tasks.	Begin with clear tasks and a weekly plan. Goal decomposition and learned estimates come in later stages.

Hierarchical optimization (Challenge 5). Goals are not equally important. We handle them in this strict order:

1. Satisfy the non-negotiable (hard) constraints (sleep $\geq 6\ \mathrm{h}$, no commitment overlaps).
2. Complete tasks before their deadlines whenever feasible.
3. When full completion is impossible, minimize unresolved work with clear priority rules, and list what is left.
4. Improve preferences and schedule stability without hurting the goals above.

2. Project Roadmap

Stage	Main focus	Expected result
1 — Core scheduling	Task input, fixed commitments, deadlines, duration estimates, conflict detection.	A weekly schedule generator that respects the hard constraints.
2 — Adaptive rescheduling	Task splitting, interruptions, schedule stability.	A system that repairs schedules after disruptions.
3 — Evaluation and usability	Automated tests, baseline comparison, student feedback.	Numbers and written usability findings.
4 — Personalization & behavioral adaptation *(future)*	Cross-week retrospective feedback (§7.8.4), psychological warm-up and fatigue modeling (§7.8.3), adaptive buffers from history.	Personalized motivation and pacing recommendations.

Stage	Main focus	Expected result
5 — Long-term goal planning *(future)*	Turn large goals and documents into milestones and weekly tasks.	A possible step to goal-based assistance.

Stages 1–3 are the core project. Stages 4–5 are longer-term options, not promised results.

9.11 Expected outcomes

The expected contribution is a personal scheduling assistant that combines **constraint-aware schedule building, user preferences and disruption recovery** in a clear academic setting. Unlike a static task list, it **places** tasks and helps when the plan becomes invalid. It avoids three over-ambitious assumptions: that every activity can be classified automatically, that task durations are always predictable, and that a student's full routine can be known without enough information. Success is measured by how well it **reduces scheduling effort, keeps user control and tells the truth about uncertainty**, not by how many tasks it puts into a calendar.

10. From design to action

The course reminds us that CT is **a strategy, not the final product**. The design phase (this proposal) must now meet the action phase: implementation, simulation and testing under real limits.

10.1 What the design phase gives us

Result	Section
A well-defined problem, with boundary, use cases and success criteria	§2
A decomposition tree with 4 perspectives and 4 modules	§3
Five patterns and four known methods to reuse	§4
A task model, a profile and a 672-slot time model	§5
A 5-step recipe with exact rules and pseudocode	§6
A system view, edge cases and design principles	§7, §8
Tests, metrics, baselines, proposed in-silico simulation (`SimulatedStudent`) and a proposed 1-week peer user study	§9

10.2 Plan for the action phase

- **Proof of concept — the core technical part first:** the time grid (M2) and the scheduling engine with local repair (M3, M4) on Minh-like data, checked against T01, T04 and T06. M1 (language model) follows. The proof of concept should show that the **core idea works**; the final product and the full evaluation come later.
- **Simulation:** run synthetic weeks with more and more tasks (§9.6).
- **Real-world limits:** model failure or delay, mobile screen, and the student's patience with data entry.

Action Phase Team Responsibility Matrix (10-Week Arc)

In accordance with our Team Contract (Lab 01) and the course project guidelines:

Weeks	Milestone & Core Deliverable	Primary Leads (Builders)	Supporting Leads (Thinkers & Leader)
W1–W3	**Core Deterministic Engine (M2, M3):** 672-slot grid, constraint validator, and 5-step greedy placement.	**Lê Minh Nhật** (Builder 2)	**Nguyễn Trọng Tiến** (Thinker 3), **Phạm Đức Minh** (Leader)
W4–W5	**Natural Language Ingestion (M1):** Structured JSON prompt pipeline, schema validation, and guardrails.	**Lê Tiến Bình** (Builder 1)	**Trần Nguyên Khánh** (Thinker 1), **Hoàng Minh Anh** (Thinker 2)

Weeks	Milestone & Core Deliverable	Primary Leads (Builders)	Supporting Leads (Thinkers & Leader)
W6–W7	**Dynamic Repair (M4) & UI/MVP Prototype:** Local repair logic, disruption simulator, and interactive user review screen.	**Nguyễn Nguyễn Gia Bảo** (Builder 3), **Lê Minh Nhật** (Builder 2)	**Hoàng Minh Anh** (Thinker 2), **Nguyễn Trọng Tiến** (Thinker 3)
W8–W9	**System Evaluation & Benchmark Execution:** Running T01–T10, sensitivity/scale tests, error taxonomy logging, and proposed user study (10–14 peer students, §9.8).	**Nguyễn Nguyễn Gia Bảo** (Builder 3), **Lê Tiến Bình** (Builder 1)	**Phạm Đức Minh** (Leader), **Trần Nguyên Khánh** (Thinker 1)
W10	**Final Engineering Defense & Handoff Pack:** Code repository cleanup, architecture pack, evaluation pack, and presentation defense.	All Members (Cross-functional verification)	**Phạm Đức Minh** (Leader & Documentation Lead)

10.3 The continuous loop in practice

Loop step	What we do
Implement a draft	Build V0 (Baseline B), then each next version.
Run implementation or simulation	Run T01–T10 and the scale and sensitivity tests.
Analyze the data to find new patterns	Use the error taxonomy; look for new patterns, such as work crowding the early week.
Repeat to refine	Change the rule or the value; run all tests again (regression).

The walk-through of §6.7 already gave the **first items** for this loop: flexible part sizes for nearly full weeks, and a soft limit on Deep Work per day (§6.9).

10.4 Open points and risks

1. **Start values are not proven.** ×1.2, ×1.35, the 30-minute dead zone, the 24-hour urgency scale, the default energy curve and the keyword table are all start values (Appendix B).
2. **The recipe is greedy.** It can fail on a nearly full week (§6.7). The first fix to test is flexible part sizes.
3. **M1 can make mistakes.** That is why the student confirms its output.
4. **Scope can grow.** Documents, long-term goals and user groups are kept out of the core on purpose (§7.2, §7.5).
5. **Students may not enter enough data.** The light setup and clear warnings reduce this risk (§8.5).

11. Conclusion

We started with a vague wish — *"help me manage my time"* — and turned it into a well-defined problem, then into a design a computer can follow. We did this with the four pillars: **decomposition** split the problem into 4 perspectives and 4 modules; **pattern recognition** gave five reusable rules and four known methods; **abstraction** kept only what changes a decision (8 task attributes and a 672-slot grid); **algorithmic design** joined deadline order, energy match, duration buffers and local repair in one 5-step recipe. Each pillar went through several attempts, and each attempt failed on a test case before a better one replaced it.

The design follows three principles: **feasibility before optimization**, **transparency before excessive automation**, and **stability before unnecessary replanning**. AI is one part of the system (reading the student's words), and not the architecture. Next, we move from design to action: build the core, test it with the plan of §9, and use what the tests show to improve the design again.

Appendix A — Terms used in this report

Term	Meaning
Schedule	The weekly plan the assistant produces.
Fixed commitment	Classes, work shifts, meetings, meals. They cannot move.

Term	Meaning
Sleep	Protected time (hard lock). Never used by the engine.
Task / macro-task / micro-task	Something to do / a large task / a piece of a macro-task, cut by **content** (for example "Code feature A").
Part (chunk)	A piece of a micro-task cut by **time** (for example 90 + 75 minutes).
Block	A part placed in the schedule: a run of consecutive slots.
Slot	A 15-minute cell of the time grid (672 per week).
Free / Occupied / Sleep / Unknown	States of a slot from the input.
Block / Break / Rest	States set by the recipe: a placed task / a 15-minute rest break / a dead zone used as active rest.
Duration buffer	The factor on an estimate: ×1.2 (load 1–2) or ×1.35 (load 3).
Planned duration	Estimate × duration buffer, rounded up to 15 minutes.
Rest break	A 15-minute break after a Deep Work block.
Deep / Medium / Shallow	Work types, with `cognitive_load` 3 / 2 / 1.
Energy level	A value from 1 to 3 on each slot.
Hard / soft constraint	A rule that must hold / a preference that improves the plan.
Urgency, importance, score	How close the deadline is / how much the task matters / importance × urgency.
Unplaced task	A task (or part) the recipe could not place.
Stability	% of unaffected blocks that stay unchanged after a repair.

Appendix B — Start values to test

Value	Used in	How we will test it
×1.2 for load 1–2; ×1.35 for load 3	Pattern 1	Compare estimated and actual time; sensitivity test (+25%, +50%).
Round up to 15 minutes	Pattern 1, time model	Check wasted time and gaps.
Maximum part 90 min; minimum part 45 min	Step 2	Check unplaced parts in nearly full weeks.
Keyword table (Deep / Medium / Shallow)	Pattern 2	Count how often students override it.
Default energy curve	Pattern 3	Compare with students' own productive hours.
10–15 min switching cost; neighbour score	Pattern 4	Check batching results.
Dead zone under 30 min	Pattern 5	Check unused gaps.
Rest break 15 min after Deep Work	Step 3	Compare Sprint and normal runs.
Urgency scale 24 hours	Step 2 score	Check order of tasks on T02, T06.
Stability goal 90%	Metrics	Measure on T04 and repeated events.
Time goal 2 s for 50 tasks	Metrics	Measure with the scale test.

Appendix C — Coverage of the project requirements (R1–R8)

ID	Requirement	Status in this proposal	Where
R1	Problem boundary	**Covered:** target user, pain point, 10 use cases, non-goals, constraints, success criteria.	§2
R2	Deterministic capability	**Designed:** the 5-step recipe is transparent and testable. To be built.	§6

ID	Requirement	Status in this proposal	Where
R3	Data / retrieval capability	**Partly:** sources are the student's commitments, the keyword table and the default energy curve. Their origin and details must be written down when we build. Document sources come in Stage 5.	§4, §5, §7.2
R4	AI-enabled capability	**Designed:** the language model reads the student's words (M1). It is justified because the language is ambiguous. To be built and tested.	§7.1
R5	Routing / orchestration	**Designed:** routing table with clarification and fallback.	§7.3
R6	End-to-end interface	**Planned:** mobile app with its own screen; the proof of concept may use a simpler one.	§7.2, §10.2
R7	Reliability and safeguards	**Planned:** checks, failure behaviour, privacy, document safety. To be written in detail.	§7.7
R8	Evaluation	**Planned:** test set, metrics, error list, improvement cycle and regression. Results come after implementation.	§9